Question 1

Part (a)

The command “file_size(pattern =”\.csv$“)” from the “phsmethods” package returns the file sizes of all .csv files in the working directory. Therefore, we know that “bbc_news_20220307_20240703.csv” is 12MB in size.

“print(summary(bbc))” prints a 1x2 tibble showing the names of each column in the bbc database, the number of rows in each column, the class type of each column, and the mode of each column. By using the “summary()” function, we can observe that it contains 35860 articles.

To find the earliest and latest dates of articles in the bbc database, I first used the package “Lubridate” to convert the pubDate values into a ddyymm_hhmmss format. I then created a new database “bbc_date” by mutating the “bbc” database with a new column containing the date in this new format, along with the year, month, and day in separate columns.

Using the year column, I was able to filter the “bbc_date” database to show the oldest article entry in 2013, and the newest article entries in 2024.

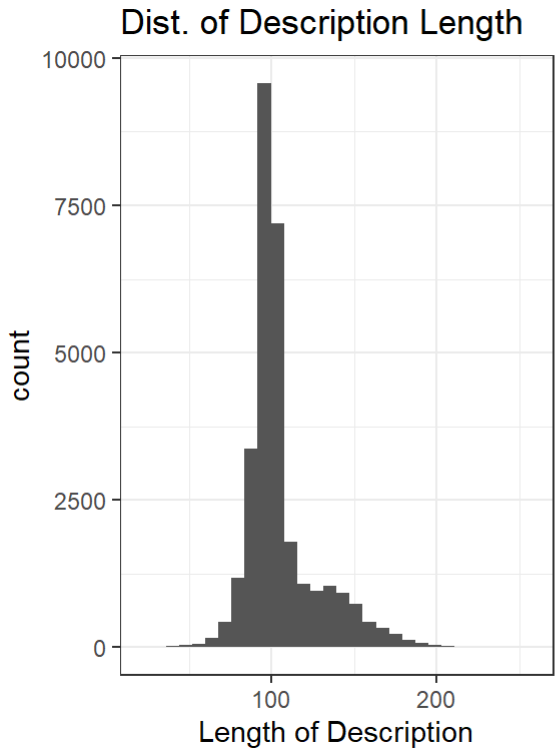
```
##     title             pubDate              guid               link
##  Length:35860       Length:35860       Length:35860       Length:35860
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##  description
##  Length:35860
##  Class :character
##  Mode  :character
```

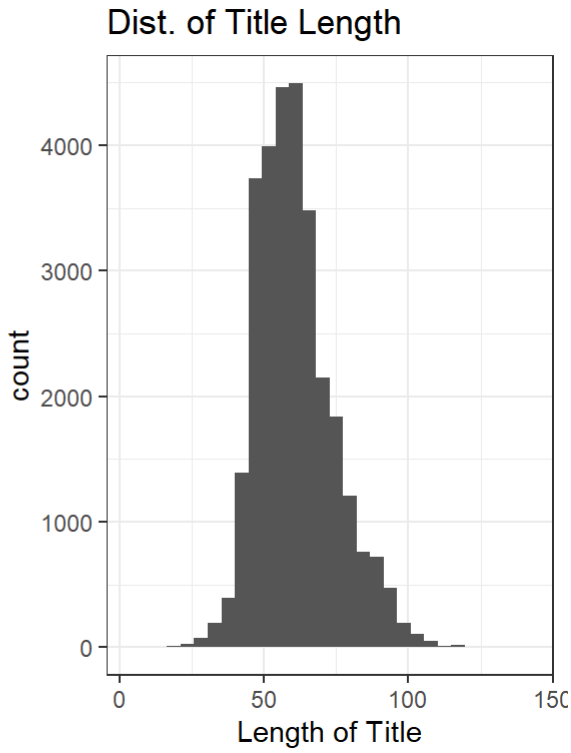
To detect duplicate or near-duplicate articles, I compared the entries for titles, descriptions, guids and links in the bbc database. Entries with identical titles, descriptions, guids or links were removed.

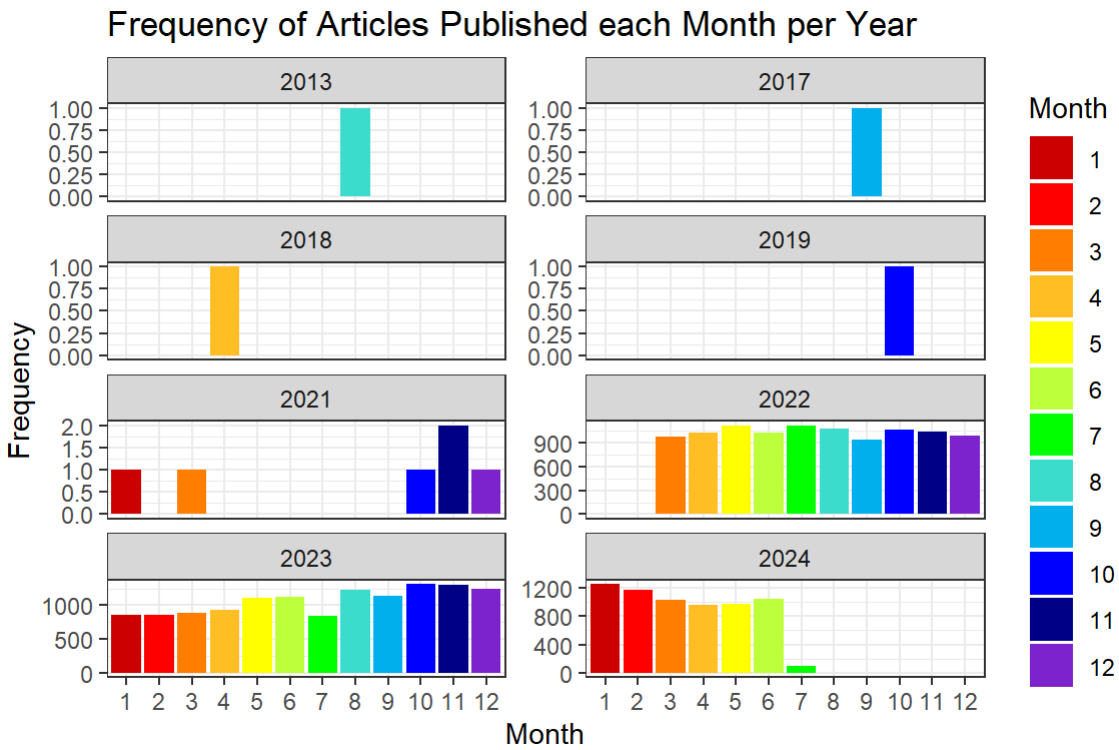
In total, 6,061 articles were removed based on this criteria.

I checked for any N/A entries in the database as well as any entries with empty strings. No empty entries for article titles or descriptions were found. I also checked the validity of any titles and descriptions that we exceedingly short, but upon checking, they were valid in spite of their length.

I checked for invalid publishing dates in the database - i.e. months that don’t exist, or days that cannot exist in any given month and year (leap years were checked for February). I found no invalid publishing dates for any article. No invalide dates were found.







Part (b)

To categorise articles by topic, I used a keyword-based classification method. To classify each article, I combined the title and link into a single text field using str_glue_data() and converted it to lowercase with str_to_lower(). I then used grepl() to search this text for topic-related keywords (e.g., “football”, “market”, “covid”, “technology”). If a keyword was found, the article was assigned to that topic. Articles without any keyword matches were placed into “Other” category.

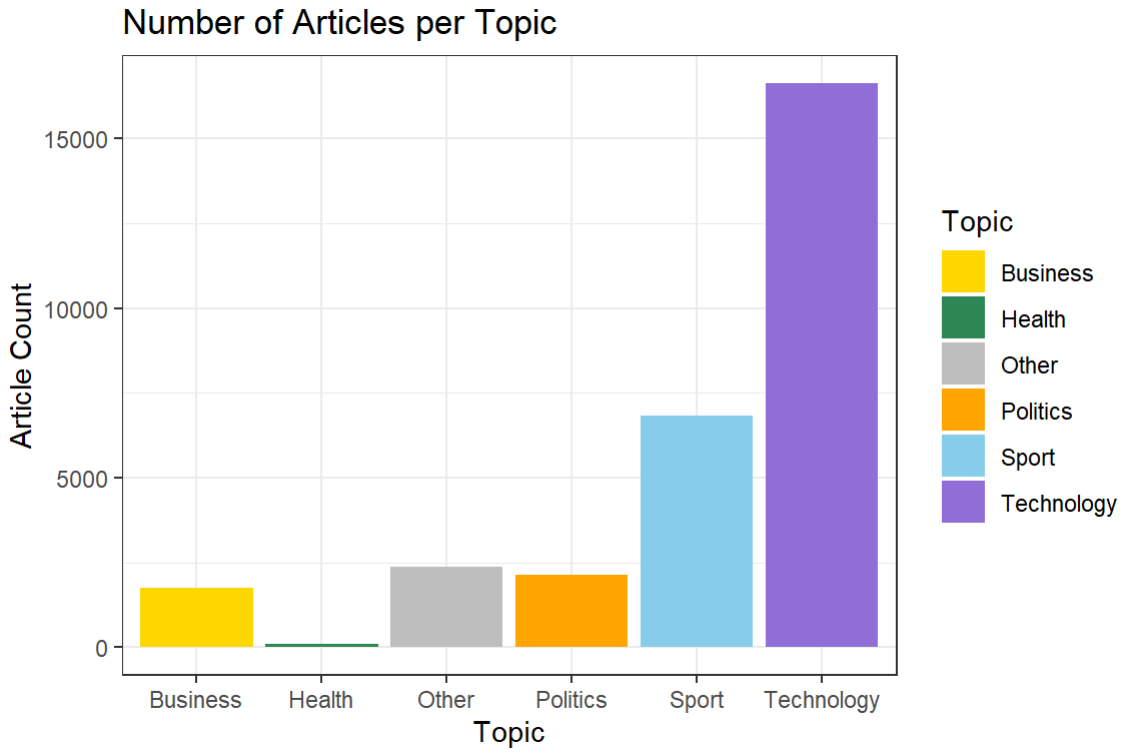
```
bbc_topics <- bbc_9 %>%
  mutate(
    text_all = str_glue("{title} {link}"),
    text_all = str_to_lower(text_all),

    topic = case_when(
      grepl("sport|football|tennis|cricket", text_all) ~ "Sport",
      grepl("business|economy|market", text_all) ~ "Business",
      grepl("politic|election|government", text_all) ~ "Politics",
      grepl("tech|technology|ai|internet|software", text_all) ~ "Technology",
      grepl("health|covid|nhs|medical|vaccine", text_all) ~ "Health",
      TRUE ~ "Other"
    )
  )

#count articles per topic
topic_counts <- bbc_topics %>%
  group_by(topic) %>%
  summarise(count = n())
topic_counts
```

topic <chr>	count <int>
Business	1745
Health	83
Other	2383
Politics	2124
Sport	6822
Technology	16642

6 rows

This bar plot visualizes how many articles fall into each topic.

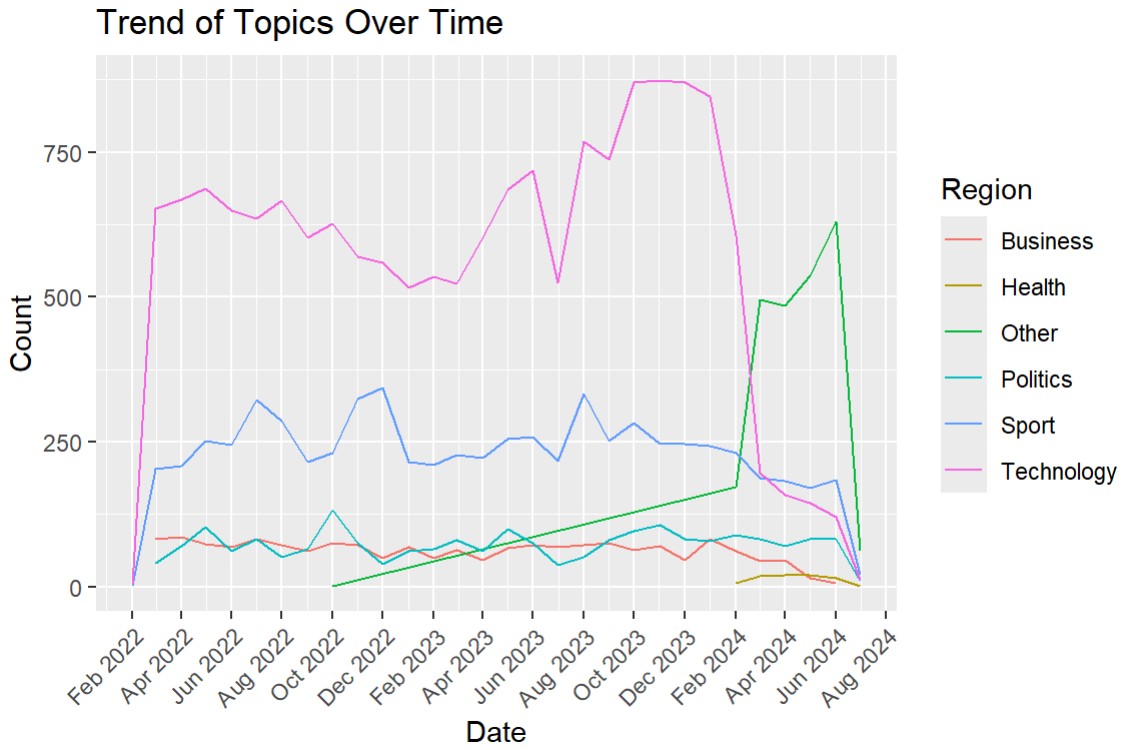
```
# Defined colors for each topic
topic_colours <- c("Sport" = "skyblue", "Politics" = "orange", "Business" = "gold", "Technolo
gy" = "mediumpurple", "Health" = "seagreen", "Other" = "grey")

ggplot(topic_counts, aes(x = topic, y = count, fill = topic)) +
  geom_bar(stat = "identity") +
  scale_fill_manual(values = topic_colours) +
  theme_bw() +
  labs(
    title = "Number of Articles per Topic",
    x = "Topic",
    y = "Article Count",
    fill = "Topic"
  )
```



We see from the above plot, that there seem to be a majority of articles in the Technology topic. The second largest topics is sport and then other. The fact that other is the third largest might indicate that we would need to use even more keywords if we want to capture the topic of all the articles.

I will now create a plot which shows the trend of topics over time. Here, I group the articles by month and count the number of articles per month. I filter every article from before 2022 since there is only very few articles from before 2022

```
# summarised how many articles appear each month for every topic
monthly_topics <- bbc_topics %>%
  mutate(date = as.Date(date, format = "%Y-%m-%d %H:%M:%S", tz = "GMT"),
         date_month = as.yearmon(date)) %>%
  filter(date_month >= as.yearmon("2022-01-01")) %>%
  group_by(date_month, topic) %>%
  summarise(count = n())

# Create the plot
ggplot(monthly_topics, aes(x = as.Date(date_month), y = count, color = topic))+
  geom_line() +
  scale_x_date(date_breaks = "2 month", date_labels = "%b %Y") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Trend of Topics Over Time",
       color = "Region",
       x = "Date",
       y = "Count")
```



From the above plot we see that the trend for Politics and Business seems to be somewhat constant without any larger spikes with Business having a small spike around Oct. 2022. Sport also seems to be constant without any clear trends. The technology topic however have a significant rise in Jul. 2023 then a sudden drop and another rise. We also see that it then drops dramatically around Jan. 2024. At the same time Technology drops the Other topic has a significant increase. This could indicate that our keywords for Tecnology does not word for the articles from 2024 and they are being marked as Other even when they should be Technology.

I will now look at the average length of the article description for each of the articles.

```
# I calculate description length for each article
bbc_topics2 <- bbc_topics %>%
  mutate(description_length = nchar(description))

# Then the average description length for each topic
topic_length_summary <- bbc_topics2 %>%
  group_by(topic) %>%
  summarise(avg_description_length = mean(description_length, na.rm = TRUE))

print(topic_length_summary)
```

```
## # A tibble: 6 × 2
##   topic      avg_description_length
##   <chr>                       <dbl>
## 1 Business                     96.6
## 2 Health                       96.2
## 3 Other                        96.9
## 4 Politics                     98.5
## 5 Sport                       135.
## 6 Technology                   96.6
```

We see that all the topics besides Sport have have a very similar average description length. This might be a standard required by the BBC. We see that sport has a significantly larger average description length than this standard. This might be because Sport articles needs to give a resumes of different matches which would

increase the length.

We now look at 10 different rows from the other category, to see if there is a pattern to the articles in this topic

```
set.seed(140)
# These articles do not contain strong topic keywords.
other_articles <- bbc_topics %>%
  ungroup() %>%
  filter(topic == "Other") %>%
  sample_n(size = 10) %>%
  select(title)

other_articles
```

title <chr>
Plane diverted to Brazil after turbulence injures 30
Michael Mosley: How the presenter transformed people's lives
Man in court over sword attack that killed boy
Russians flock to see captured Western tanks
People left dangling upside down on amusement park ride
'I could not protect her' - a dad mourns his daughter killed in Channel
Hundreds with gynaecological cancer feel ignored
Parasite found in south Devon water prompts urgent warning
Isfahan - strategic Iranian city where explosions heard
Watch: St Patrick's Day celebrated across the world

1-10 of 10 rows

From the above sample we see that we have many different topics, there is articles about war in the Middle-East, Ukraine-Russia, Diseases etc. Some of these articles would definitely fit into the categories we defined earlier but however still ends up in the Other topic. This indicates that if we wanted to categories all the articles, we would need more keywords for the different topics.

Part (c)

Firstly, we want to find meaningful regions for the articles based on the data available in the dataset. We notice that it seems like the “guid” variable is the only variable indicating which region the article is regarding. We however also notice that it only seems to be the news articles which include this region in the guid. Here, the guid variable have one of 2 pattern for the news article: 1) “…news/world-‘country’-‘ID’” or 2) “…news/uk-‘uk area’-‘ID’”. We therefor want to create a function which given the guid for the world format return the country and for the uk format returns uk and the area in the uk. Below I have created this function, which uses a regular expression to locate the relevant part of the guid and the returns the region.

```
# Creating function to extract regions from guid
extract_regions <- function(guid) {

  # Extracting relevant part of guid using regular expression
  x <- gsub(".*/news/([a-z-]+)-[0-9]+.*", "\\1", guid)

  # Splitting the region part of guid into individual strings based on "-"
  x <- strsplit(x, "-")[[1]]

  if(x[1] == "world"){ # For guid with world remove the world string
    x = x[-1]
  }
  if(x[1] == "middle" |x[1] == "latin") { # For guid with "middle-east" or "latin-america" re
gion combine the separated strings
    return(paste(x[1:2], collapse = " "))
  }
  if(x[1] == "south"){ # For guid from south asia return only asia
    return("asia")
  }
  #if(length(x) > 1 & x[2] == "politics"){ # For guid with politics in region only return reg
ion
  #  return(x[1])
  #}
  if(x[1] == "us"){ # For guid from with "us-canada" region combine the seperated strings wit
h an &
    return(paste(x[1:2], collapse = " & "))
  }
  return(x)
}
```

Not all the regions are on the same format as shown by all the special cases in the above function. Here, we had to look at the possible regions in the guid and create the function based on these. As seen above we denote an article from “south-asia” to be from “asia” that is beacause only 5 articles are from “south-asia” and it therefor did not make sense to look at this as an independent region.

Now we want to apply the function to the data. Here, we first select the relevant articles to use the function on. As mentioned, it is only the news articles which shows the above mentioned region pattern in the guid. So below we select all the news articles. Some of the news articles just had “world” as their region in the guid. This is not a meaningful region and these are therefor dropped. Furthermore, for some of the articles from Scotland, Scotland would be mentioned twice in the region. This is changed to only one mentioning of Scotland.

```
# Selecting only the articles which inform about region in guid
bbc_data_select <- bbc_topics %>%
  ungroup() %>%
  filter(gsub("news/world", "", guid) != guid | gsub("news/uk-", "", guid) != guid) %>%
  filter(!grepl("world-[0-9]", guid)) %>%
  mutate(guid = gsub("politics-|business-", "", guid)) %>%
  mutate(guid = gsub("scotland-scotland-", "scotland-", guid))
```

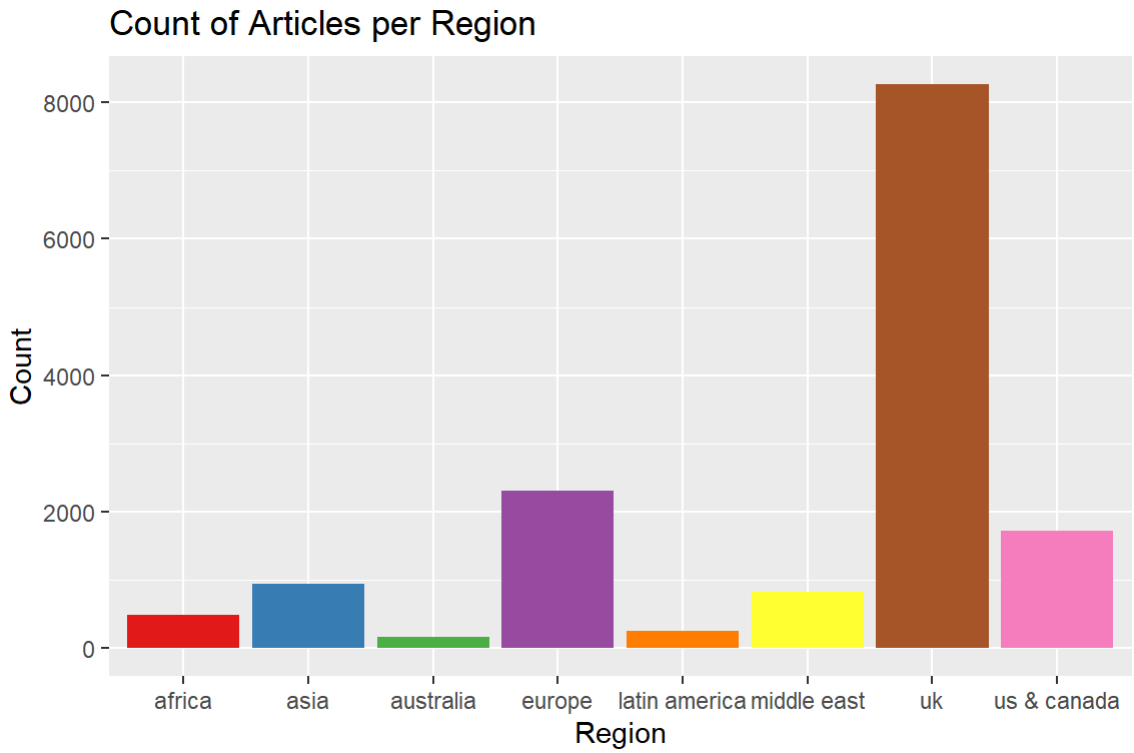
Now we can apply the function to the selected articles

```
# Get the region from the selected articles
regions = lapply(bbc_data_select$guid, extract_regions)
```

We now divide the region into two parts: a broad region (i.e. Africa, Asia, Australia, Europe, Latin America, Middle East, UK, US & Canada) and a precise region (i.e. the local UK region for the articles from the UK)

```
# Format the regions in desired manner
bbc_data_region <- bbc_data_select %>%
  mutate(region_broad = sapply(regions, function(x) x[1])) %>%
  mutate(region_precise = sapply(regions, function(x) if (length(x)>1){return(paste(x[2:lengt
h(x)], collapse = " "))}else{return("None")}))
```

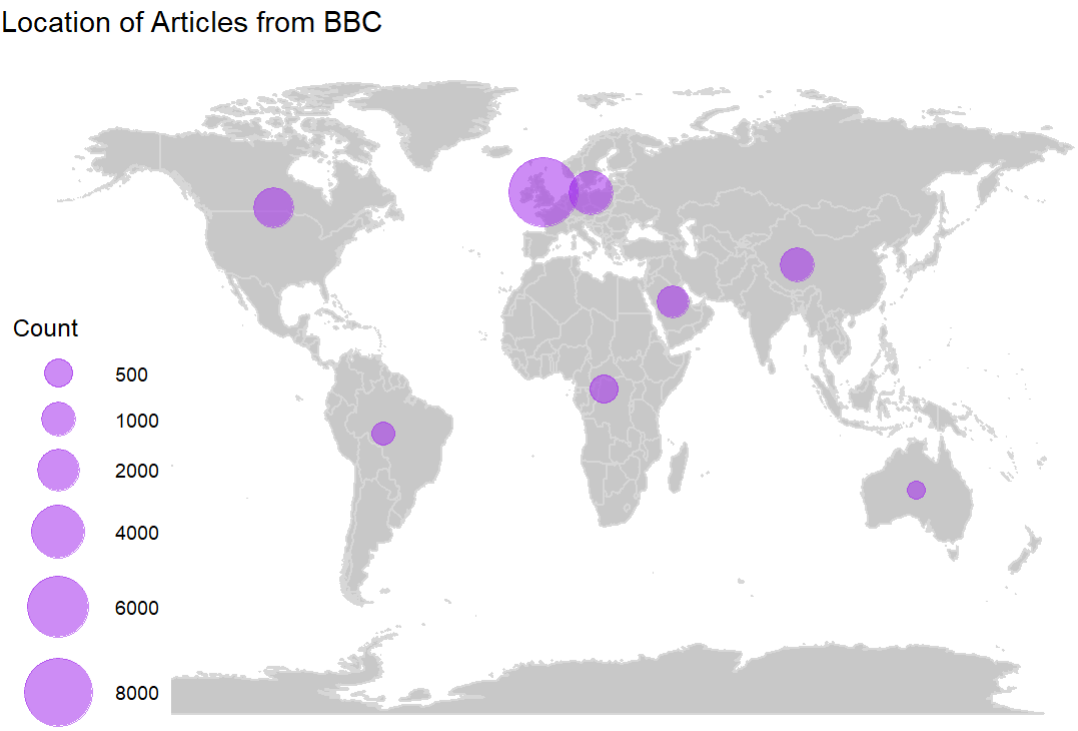
We now create a bar plot of the frequency of articles for the broad region (similar to the barplot over topics in exercise 1.b):



From the above plot we see that a majority of the articles are regarding news from the UK. This is not surprising since the BBC is a UK news company and receive funding from the UK government issued TV licence. After the UK we see that the two most popular regions are “europe” and “us & canada” which again is not surprising, since western media often focus on topics from western countries.

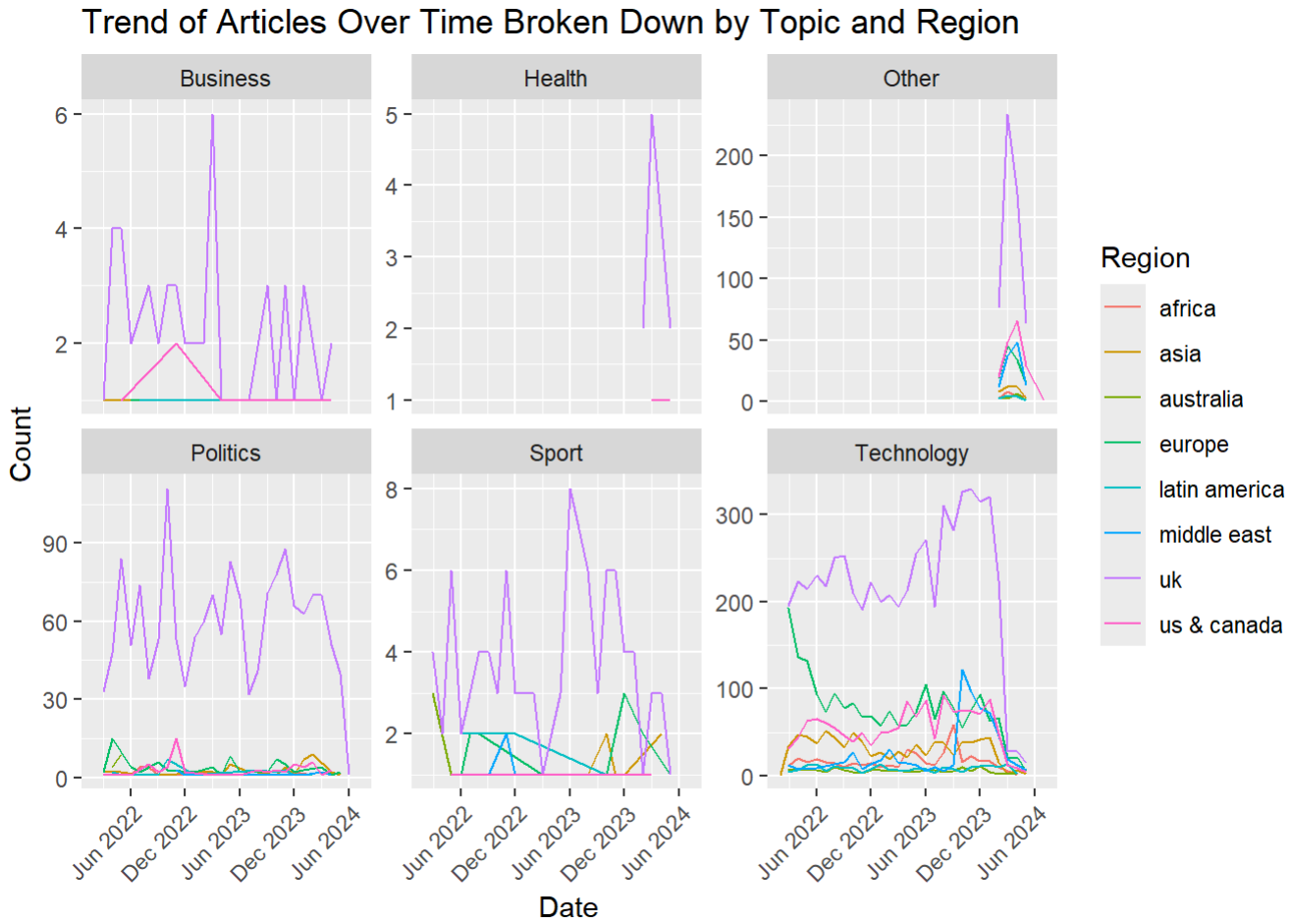
To give a more visual representation of the regions from the different BBC news articles we will create a plot with a world map with a dot on the map for each region. The size of the regions is indicating the count of articles from that region. The location of the dot is made to approximate the center of the region.

```
bbc_data_region_count = bbc_data_region_count %>%
  mutate(lat = c(2, 35, -25, 54, -10, 25, 54, 50),
         lon = c(20, 90, 133, 15, -60, 45, -2, -100))

world_map <- map_data("world")

world_basemap <- ggplot() +
  borders("world", colour = "gray85", fill = "gray80")+
  theme_map()

world_basemap +
  geom_point(data = bbc_data_region_count, aes(x = lon, y = lat, size = count),
             colour = 'purple', alpha = .5) +
  scale_size(range = c(3, 12), breaks = c(500, 1000, 2000, 4000, 6000, 8000)) +
  labs(title = "Location of Articles from BBC", size = "Count")
```



The above plot makes it even more clear that the BBC is mainly focused on news from areas close to the UK.

Now we want to create a time series plot which shows article trends over time broken down by region and topic. The date element from the dataset gives the exact time, down to the second, when the articles were published. Using this exact time to show trends would not be beneficial since we would never have two articles published the exact same time. Instead we group the articles based on published year and month and look over the trends for the different months. From the analysis in part a that a very few number of articles were published before 2022. We remove all these articles since they would seem as outliers in the time series. The code for the below plot is left out as it is very similar to the code used in exercise 1.b to create the plot of topic trends over time. Here, we just color region instead of topic and do a facet_wrap() of topic.



From the above plot we see that we have so few articles belonging to the topics Business, Health and Sport that it is difficult to say anything about their trends over time. We also see that these non of the topics really have that many articles from 2024 however the Other topic only have articles from 2024 indicating that the method used to identify the topics might not work so well for the articles from this period.

If we look at the Politics topic then we see that it is dominated with articles from the UK. Again, this is to be expected since the BBC is a UK based news company. The trend of news article regarding Politics seems to be very volatile with sudden peaks and sudden drops. The field of politics are also very unpredictable and we often see sudden stories or scandals gaining popularity in the media. However, it would probably be expected that the articles regarding news would be increasing up to a general election. The last general election in the UK was on the 4th of June 2024. We however see, that the number of articles regarding politics are dropping in 2024. This however might be due to an issue with finding topics for articles in this period, as mentioned above.
s

If we look at the topic technology, then we see that this is clearly the topic with most articles. Again, this topic is dominated by articles from the UK but here we can also see some trends from other regions. The trend for technology articles over time in the UK seems to steadily rise, which would correspond with the extra public focus on technology such as generative AI after the release of ChatGPT. We however see, that new articles regarding technology in Europe peaked in early to mid 2022 and have been somewhat decreasing since. This is contrary to the trend in the UK. We also see a very sudden increase in Technology articles from the Middle East in the fall 2023. This is the exact timing of the reignition of the Israel-Palestine conflict which since gained a significant amount of media attention. Here, it would however have expected to cause an increase in Politic articles from the Middle-East so maybe this spike is caused by something else or there is some patterns in these articles which gets them marked as technology.

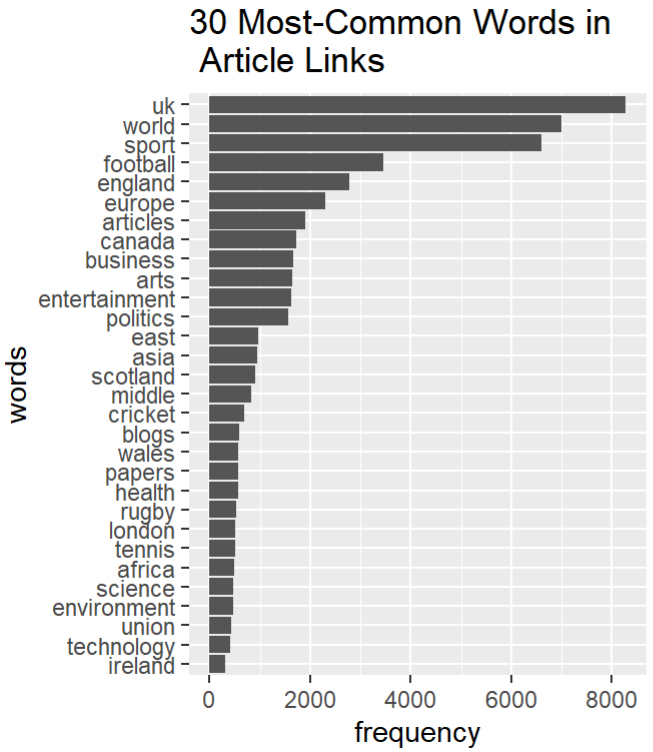
Part (d)

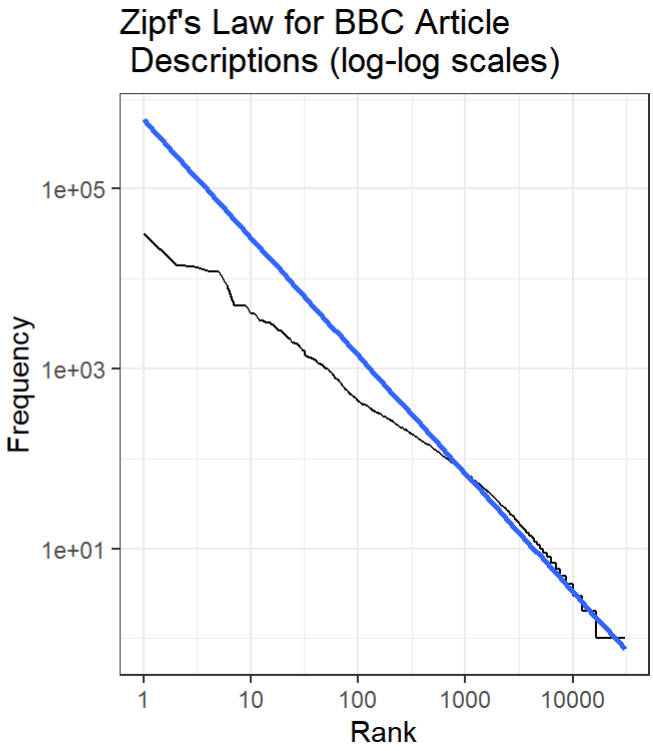
Part (d) asks us to “distinguish betwen primary research categories”. This wording is somewhat ambiguous as it is not clearly distinct from the “topics” that were distinguished in Part (c). We can therefore define our own categories and assess their usefullness in our analysis. Stopwords were used only in the categorisation of articles, not in the zipf analysis.

The following code chunk categorises the articles into the following categories:

1. from the .co.uk or .com bbc domain,
2. long or short in the length of their description,
3. geographic area.

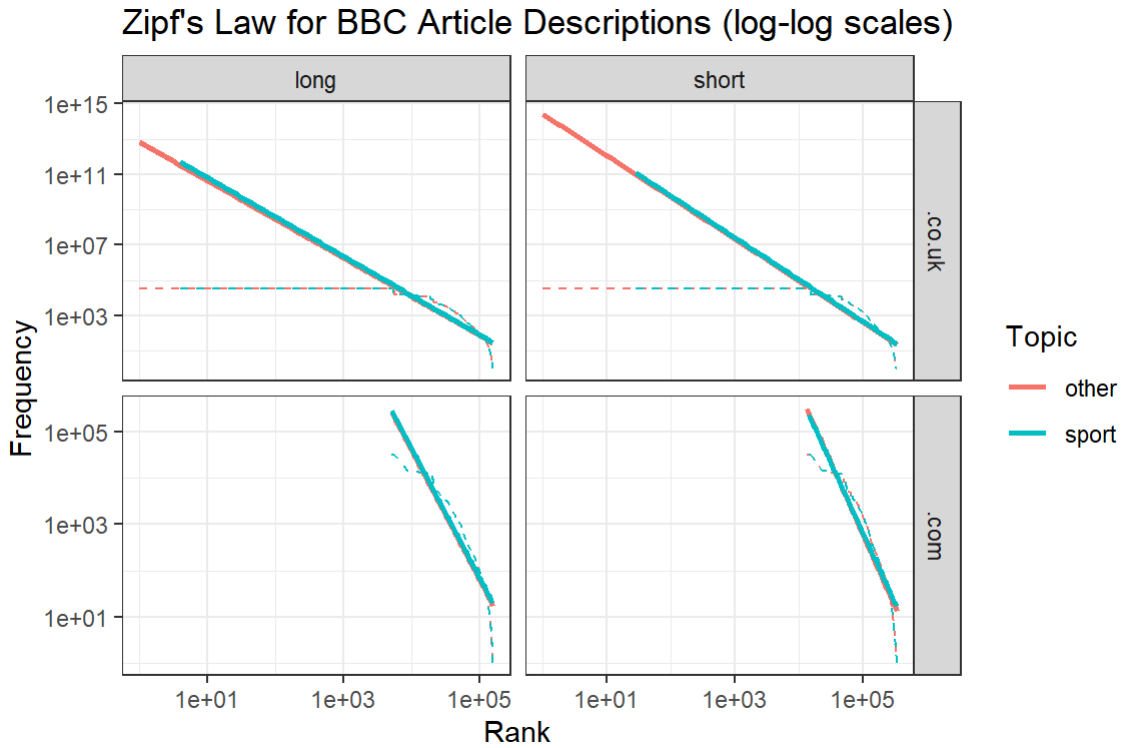
Something to note: the text “at_campaign=karanga” in some of the article links, refers to no singular topic. Therefore, “at”, “campaign”, and “karanga” (along with other words that provide no analytical insights) have been omitted from analysis.





The Zipf’s Law graph for Part (d) is approximate to what we would expect in that the log-log graph shows an inverse relationship between rank and frequency (as demonstrated by the linear fit line). However, there is some “stepping” behaviour towards the bottom and top of the x-axis, indicating that words of the highest and lowest ranks occur with a similar frequency across the corpus.

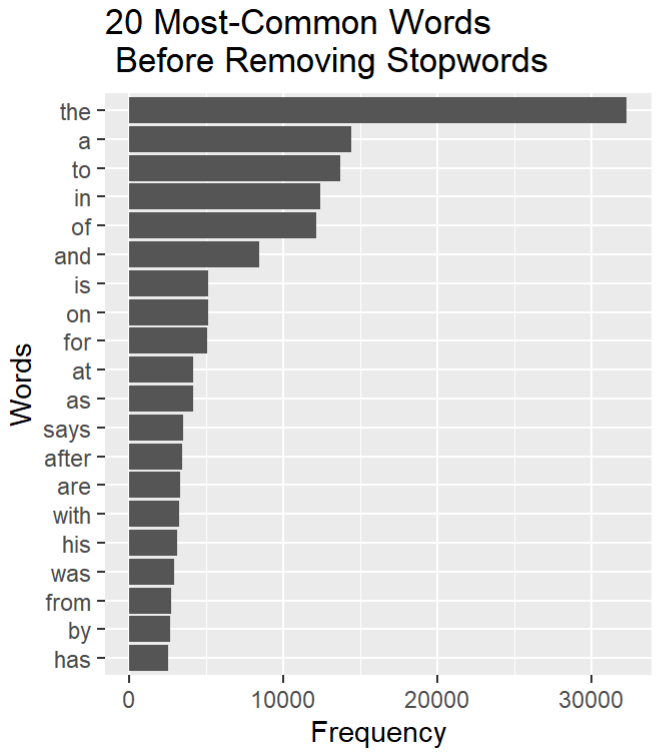
The intercept at the y-axis is very high, indicating that there is a lot of repition of words within the corpus, and the slope is quite steep, indicating that the article descriptions contain a limited range of unique words.

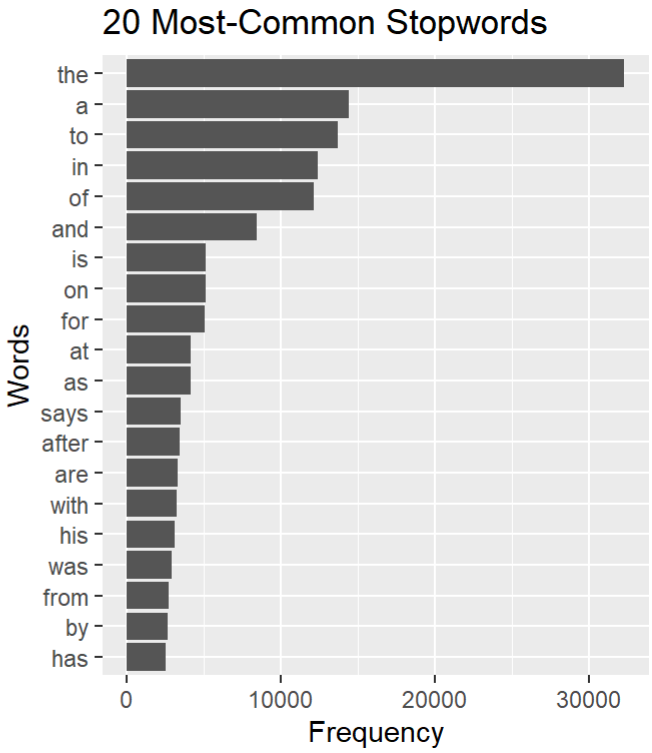


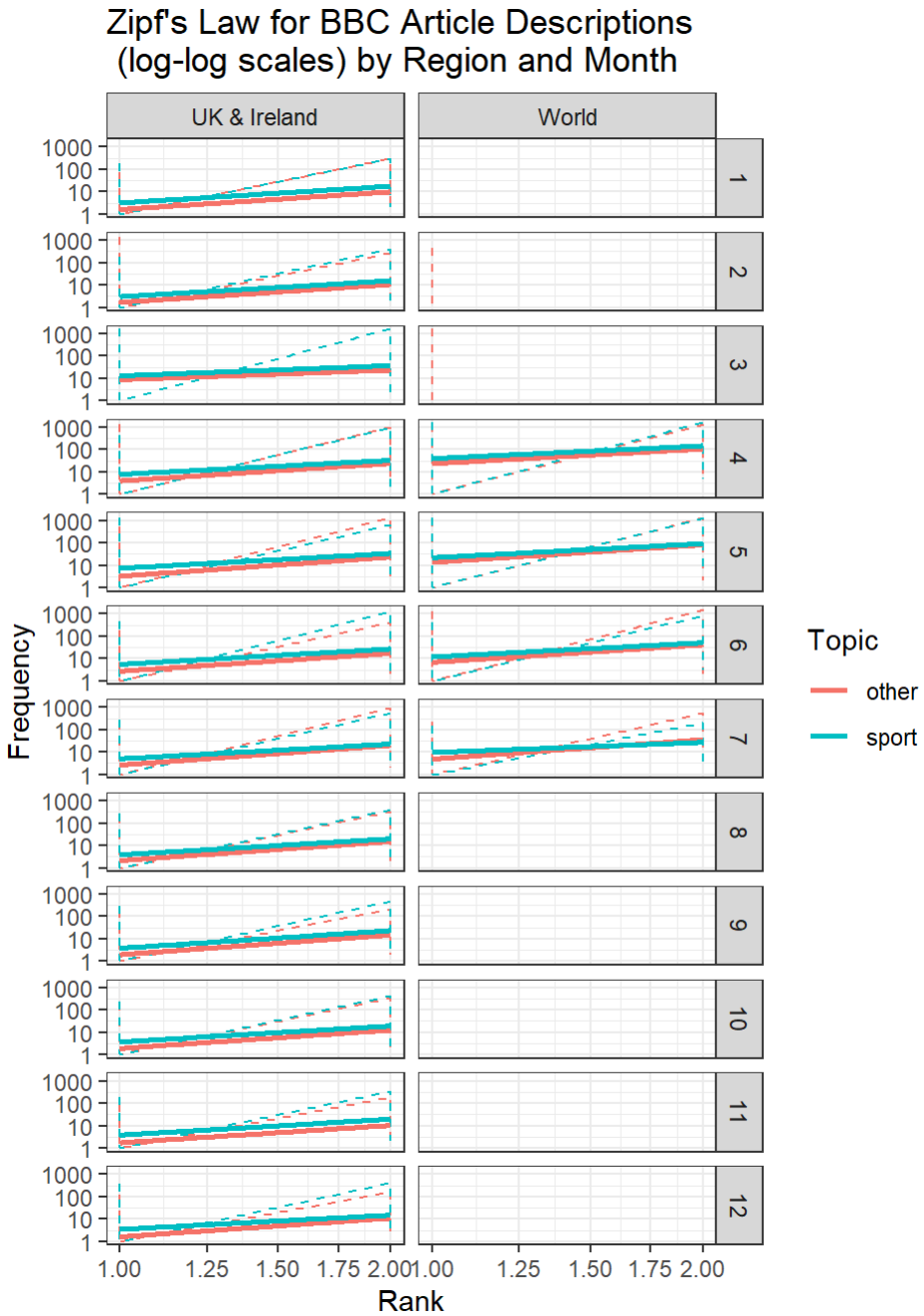
Part (e)

Part (e) is not specific in wether it requires stopwords to be left in or taken out. For this reason, I started answering the question by leaving stopwords in. However, I found that only stopwords were shown as the 20 most-frequent words across the corpus. Therefore, I had to filter out stopwords to perform any meaningful analysis.

Articles were compared using the month in which they were published, wether the were on the topic of sport, and wether they were from the region of the UK and Ireland or from the rest of the world.







I used both the Loughran and Bing methods to identify the sentiment of words in the articles' description. I used both to maximise the number of words recognised as carrying a sentiment, thus improving the potential for analysis.

Overall, the most common sentiment was negative, followed by positive, then litigious, then constrained, and then superfluous as the least common sentiment. This was consistent with my expectations, since news articles frequently pertain to devestating events and global/national concerns.

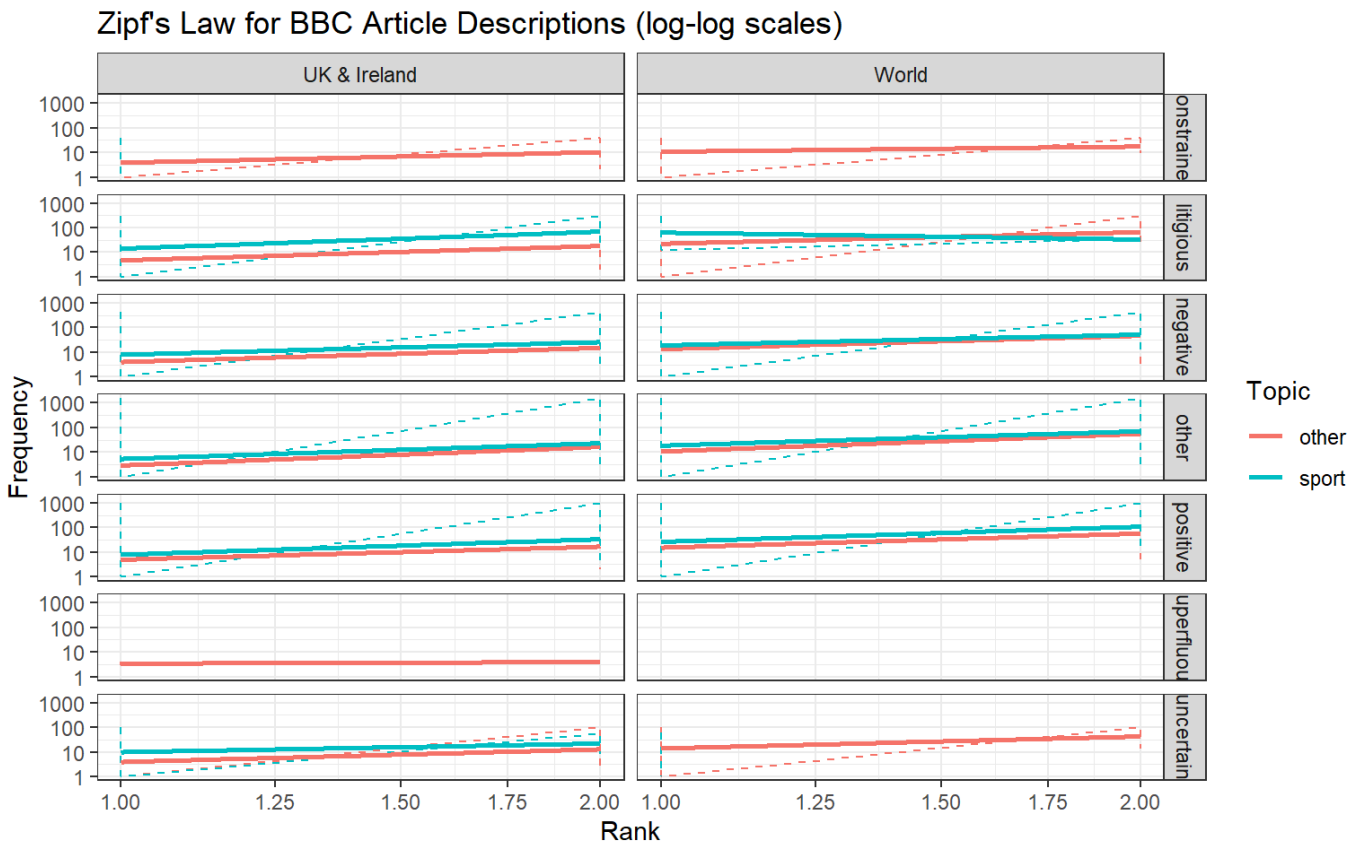
```
## Number of words with a positive sentiment:  2109 .
```

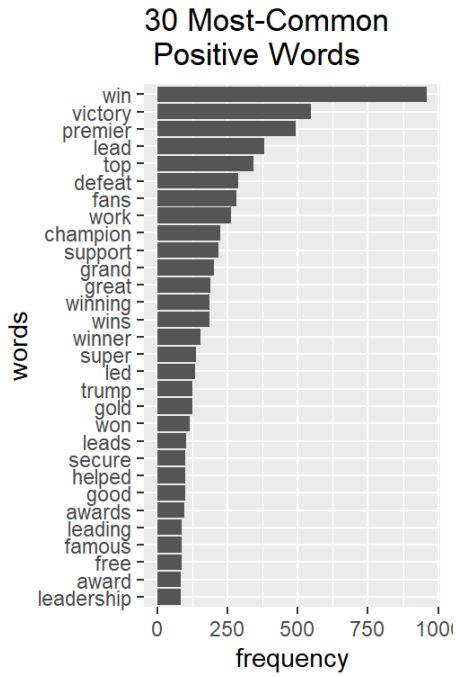
```
## Number of words with a negative sentiment:  4541 .
```

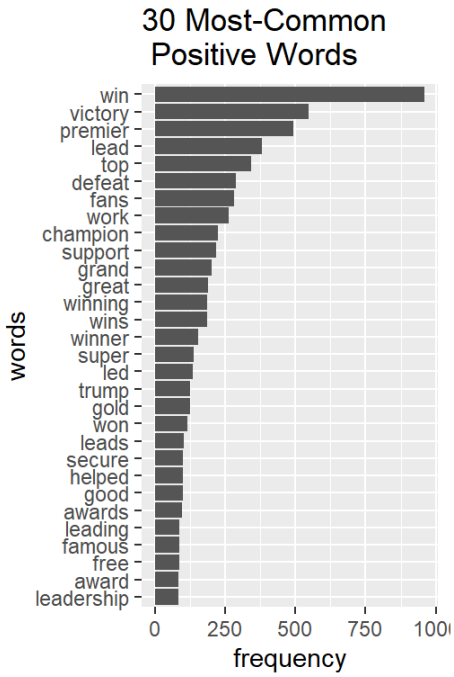
```
## Number of words with a litigious sentiment:  196 .
```

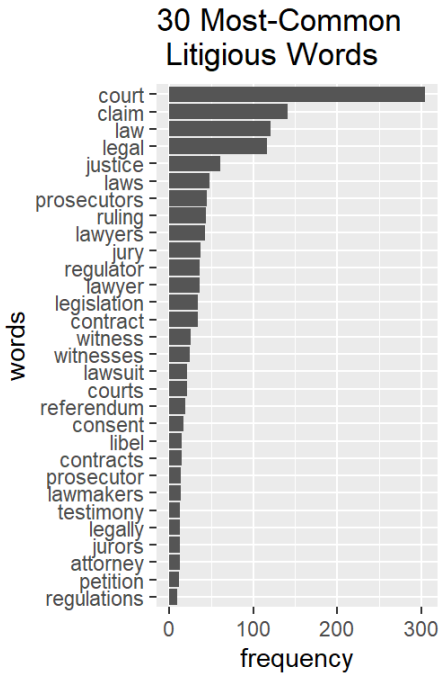
```
## Number of words with a constrained sentiment:  111 .
```

```
## Number of words with a superfluous sentiment:  5 .
```









Question 2

Part (a)

Data Loading and Preparation

```
# Load all PLYM files
plym_2020 <- read_csv("data/polution/PLYM_2020.csv", skip = 4) %>% mutate(site = "plym", Year
= "2020")

plym_2021 <- read_csv("data/polution/PLYM_2021.csv", skip = 4) %>% mutate(site = "plym", Year
= "2021")

plym_2022 <- read_csv("data/polution/PLYM_2022.csv", skip = 4) %>% mutate(site = "plym", Year
= "2022")

plym_2023 <- read_csv("data/polution/PLYM_2023.csv", skip = 4) %>% mutate(site = "plym", Year
= "2023")

plym_2024 <- read_csv("data/polution/PLYM_2024.csv", skip = 4) %>% mutate(site = "plym", Year
= "2024")

plym_2025 <- read_csv("data/polution/PLYM_2025.csv", skip = 4) %>% mutate(site = "plym", Year
= "2025")

# Load all PLYR files
plyr_2020 <- read_csv("data/polution/PLYR_2020.csv", skip = 4) %>% mutate(site = "plyr", Year
= "2020")

plyr_2021 <- read_csv("data/polution/PLYR_2021.csv", skip = 4) %>% mutate(site = "plyr", Year
= "2021")

plyr_2022 <- read_csv("data/polution/PLYR_2022.csv", skip = 4) %>% mutate(site = "plyr", Year
= "2022")

plyr_2023 <- read_csv("data/polution/PLYR_2023.csv", skip = 4) %>% mutate(site = "plyr", Year
= "2023")

plyr_2024 <- read_csv("data/polution/PLYR_2024.csv", skip = 4) %>% mutate(site = "plyr", Year
= "2024")

plyr_2025 <- read_csv("data/polution/PLYR_2025.csv", skip = 4) %>% mutate(site = "plyr", Year
= "2025")

# Combine all data
air_data <- bind_rows(
  plym_2020, plym_2021, plym_2022, plym_2023, plym_2024, plym_2025,
  plyr_2020, plyr_2021, plyr_2022, plyr_2023, plyr_2024, plyr_2025
)

# Convert Date column - format is "dd-mm-yyyy" with dashes
air_data <- air_data %>%
  mutate(DateTime = as.POSIXct(Date, format = "%d-%m-%Y"))

# Convert to long format
value_cols <- c(3, 6, 9, 12, 15, 18, 21, 24, 27, 30)
```

```
airdata_longer <- air_data %>%
  pivot_longer(
    cols = value_cols,
    names_to = "type",
    values_to = "value"
  ) %>%
  mutate(Date = as.Date(Date, format= "%d-%m-%Y ")) %>%
  select(Date, site, type, value)
```

Part (b)

Selected Pollutants for Analysis

For this analysis, we focus on **6 key pollutants**: - **NO** (Nitric oxide): Primary emission from vehicles - **NO2** (Nitrogen Dioxide): Major traffic pollutant - **NOx** (Nitrogen oxides): Total nitrogen oxides - **PM2.5**: Fine particulate matter, health hazard - **PM10** (Non-volatile): Larger particulate matter - **O3** (Ozone): Secondary pollutant

```
# Extract key pollutant columns by position
# Columns: 3=Nitric oxide, 6=Nitrogen dioxide, 9=NOx, 12=Non-volatile PM10,
#          18=Ozone, 21=PM2.5
pollutants_data <- air_data %>%
  select(DateTime, site, Year,
         NO = 3,           # Nitric oxide
         NO2 = 6,          # Nitrogen dioxide
         NOx = 9,          # Nitrogen oxides as NO2
         PM10 = 12,        # Non-volatile PM10
         O3 = 18,          # Ozone
         PM2.5 = 21) %>%   # PM2.5 particulate matter
  pivot_longer(cols = c(NO, NO2, NOx, PM10, O3, PM2.5),
               names_to = "Pollutant",
               values_to = "Concentration")

print(paste("Number of rows in pollutants_data:", nrow(pollutants_data)))
```

```
## [1] "Number of rows in pollutants_data: 610848"
```

```
print(paste("Unique pollutants:", paste(unique(pollutants_data$Pollutant), collapse = ", ")))
```

```
## [1] "Unique pollutants: NO, NO2, NOx, PM10, O3, PM2.5"
```

Numerical Summaries

```
# Calculate comprehensive statistics for each pollutant
summary_stats <- pollutants_data %>%
  group_by(Pollutant) %>%
  summarise(
    Mean = round(mean(Concentration, na.rm = TRUE), 2),
    Median = round(median(Concentration, na.rm = TRUE), 2),
    Min = round(min(Concentration, na.rm = TRUE), 2),
    Max = round(max(Concentration, na.rm = TRUE), 2),
    SD = round(sd(Concentration, na.rm = TRUE), 2),
    Missing_Values = sum(is.na(Concentration)),
    Total_Obs = n(),
    Percent_Missing = round(100 * Missing_Values / Total_Obs, 2)
  )

kable(summary_stats, caption = "Summary Statistics for Selected Pollutants (µg/m³)")
```

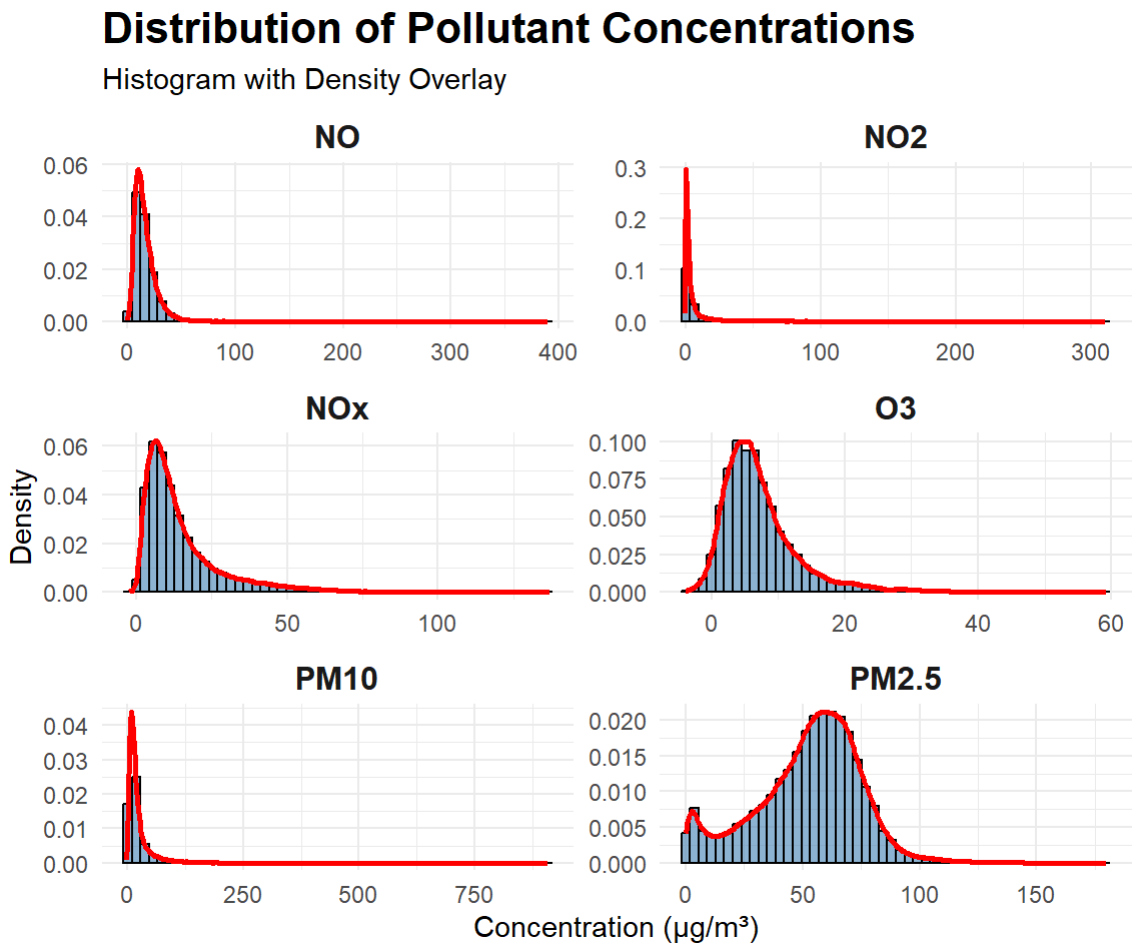
Summary Statistics for Selected Pollutants (µg/m³)

Pollutant	Mean	Median	Min	Max	SD	Missing_Values	Total_Obs	Percent_Missing
NO	15.68	13.40	-0.60	389.92	10.24	51465	101808	50.55
NO2	5.87	1.81	-0.87	310.36	14.05	8798	101808	8.64
NOx	14.76	10.48	-2.24	137.21	13.28	8818	101808	8.66
O3	7.19	6.00	-3.90	59.20	5.76	90630	101808	89.02
PM10	23.78	13.88	-1.89	907.43	32.50	8810	101808	8.65
PM2.5	52.55	55.62	0.00	179.86	22.43	52052	101808	51.13

Graphical Summaries

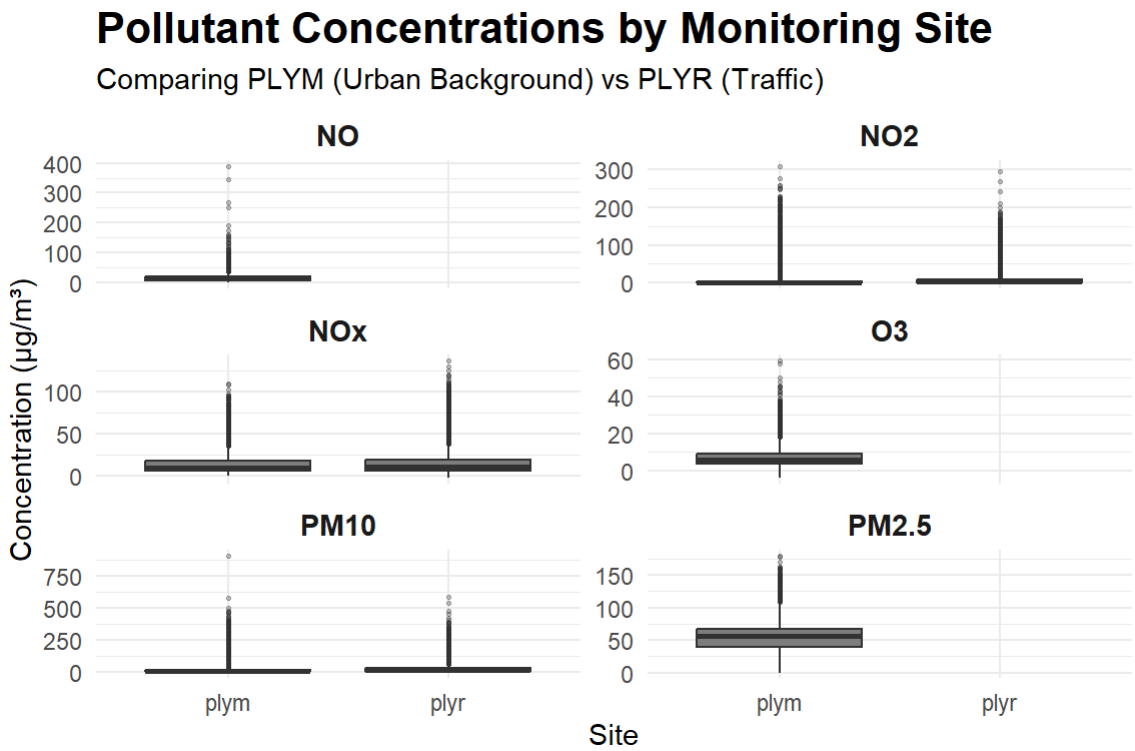
1. Histograms and Density Plots

```
# Histogram with density overlay for each pollutant
ggplot(pollutants_data %>% filter(!is.na(Concentration)),
       aes(x = Concentration)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50,
                 fill = "steelblue", alpha = 0.6, color = "black") +
  geom_density(color = "red", linewidth = 1) +
  facet_wrap(~Pollutant, scales = "free", ncol = 2) +
  theme_minimal() +
  labs(title = "Distribution of Pollutant Concentrations",
       subtitle = "Histogram with Density Overlay",
       x = "Concentration (µg/m³)",
       y = "Density") +
  theme(plot.title = element_text(size = 16, face = "bold"),
        strip.text = element_text(size = 12, face = "bold"))
```



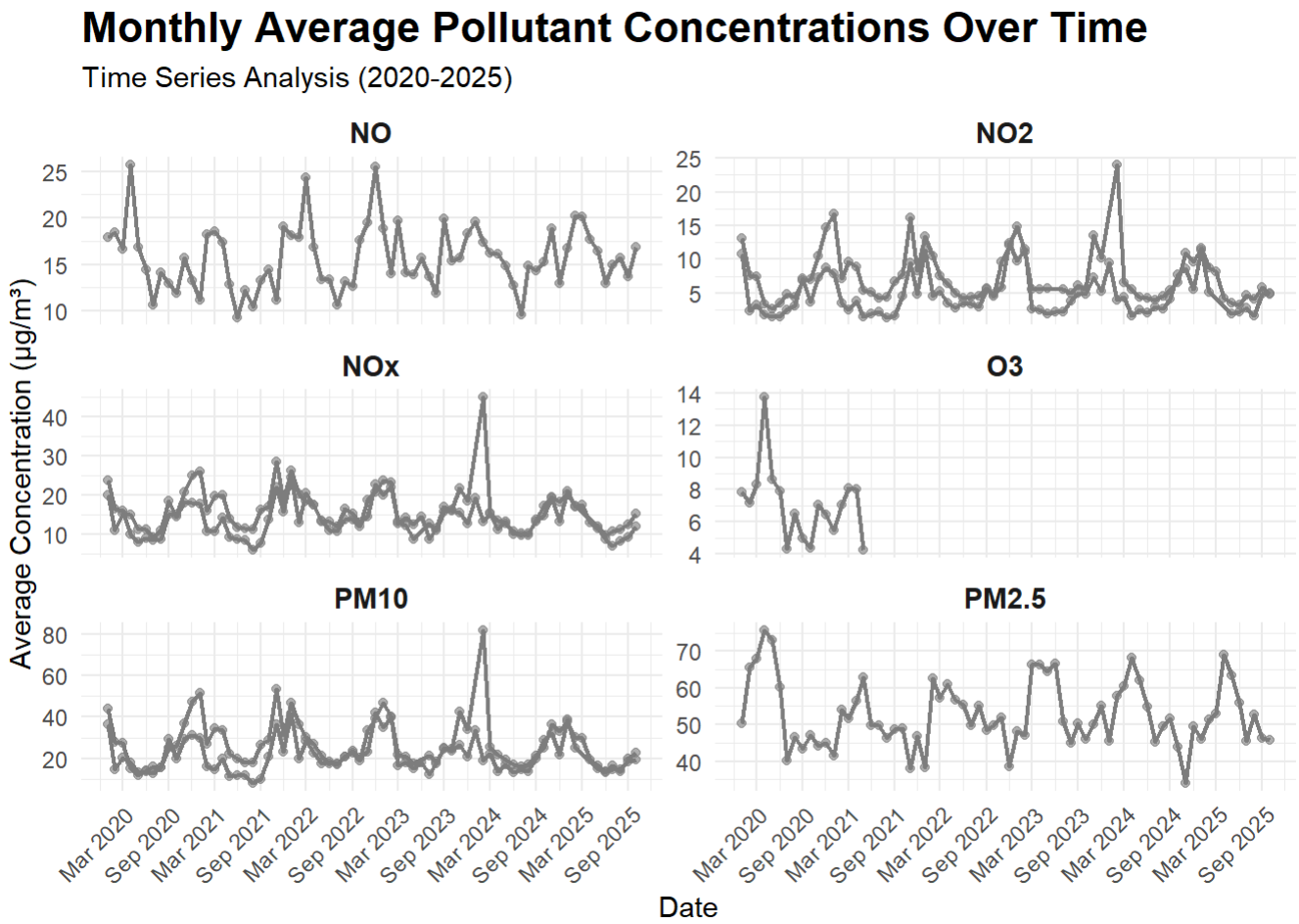
2. Boxplots by Pollutant

```
# Boxplot comparing sites for each pollutant
ggplot(pollutants_data %>% filter(!is.na(Concentration)),
       aes(x = site, y = Concentration, fill = site)) +
  geom_boxplot(outlier.alpha = 0.3, outlier.size = 0.5) +
  facet_wrap(~Pollutant, scales = "free_y", ncol = 2) +
  scale_fill_manual(values = c("PLYM" = "#66c2a5", "PLYR" = "#fc8d62")) +
  theme_minimal() +
  labs(title = "Pollutant Concentrations by Monitoring Site",
       subtitle = "Comparing PLYM (Urban Background) vs PLYR (Traffic)",
       x = "Site",
       y = "Concentration (µg/m³)") +
  theme(plot.title = element_text(size = 16, face = "bold"),
        strip.text = element_text(size = 11, face = "bold"))
```



3. Time Series Plots

```
# Monthly average time series
monthly_avg <- pollutants_data %>%
  filter(!is.na(Concentration), !is.na(DateTime)) %>%
  mutate(Year = year(DateTime),
         Month = month(DateTime),
         YearMonth = make_date(Year, Month, 1)) %>%
  filter(!is.na(YearMonth)) %>%
  group_by(YearMonth, Pollutant, site) %>%
  summarise(Avg_Concentration = mean(Concentration, na.rm = TRUE),
            .groups = 'drop') %>%
  filter(!is.na(Avg_Concentration), is.finite(Avg_Concentration))

ggplot(monthly_avg, aes(x = YearMonth, y = Avg_Concentration, color = site)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.5, alpha = 0.6) +
  facet_wrap(~Pollutant, scales = "free_y", ncol = 2) +
  scale_color_manual(values = c("PLYM" = "#1b9e77", "PLYR" = "#d95f02")) +
  scale_x_date(date_breaks = "6 months", date_labels = "%b %Y") +
  theme_minimal() +
  labs(title = "Monthly Average Pollutant Concentrations Over Time",
       subtitle = "Time Series Analysis (2020-2025)",
       x = "Date",
       y = "Average Concentration (µg/m³)",
       color = "Site") +
  theme(plot.title = element_text(size = 16, face = "bold"),
        strip.text = element_text(size = 11, face = "bold"),
        axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = "bottom")
```



Comments and Interpretation

General Patterns and Trends

The analysis reveals declining trends in most pollutants from 2020 to 2023, likely influenced by COVID-19 restrictions and subsequent recovery. NO2 and NOx show moderate to high concentrations with significant variation, while PM2.5 and PM10 display lower average concentrations with notable episodic spikes. O3 exhibits distinct seasonal patterns with elevated concentrations during summer months due to photochemical formation.

Differences Between Sites

PLYR (traffic site) consistently shows higher NO, NO2 and NOx concentrations compared to PLYM (urban background), clearly demonstrating the impact of local traffic emissions. PM concentrations are relatively similar between sites, suggesting regional rather than local sources. O3 patterns differ between sites due to complex photochemical interactions and NOx titration effects at the traffic location.

Outliers and Data Issues

Boxplots reveal numerous outliers for all pollutants, representing both genuine pollution episodes and potential measurement anomalies. Clear seasonal variations are evident, with winter months showing elevated NO2 and PM levels, while summer months exhibit higher O3 concentrations. The mix of data quality codes (Ratified, Provisional, As Supplied) should be considered when interpreting extreme values.

Conclusion

Plymouth's air quality data shows significant traffic impacts at roadside locations, temporal improvements over the study period, and strong seasonal influences. Episodic pollution events and data quality variations require careful consideration in air quality management strategies.

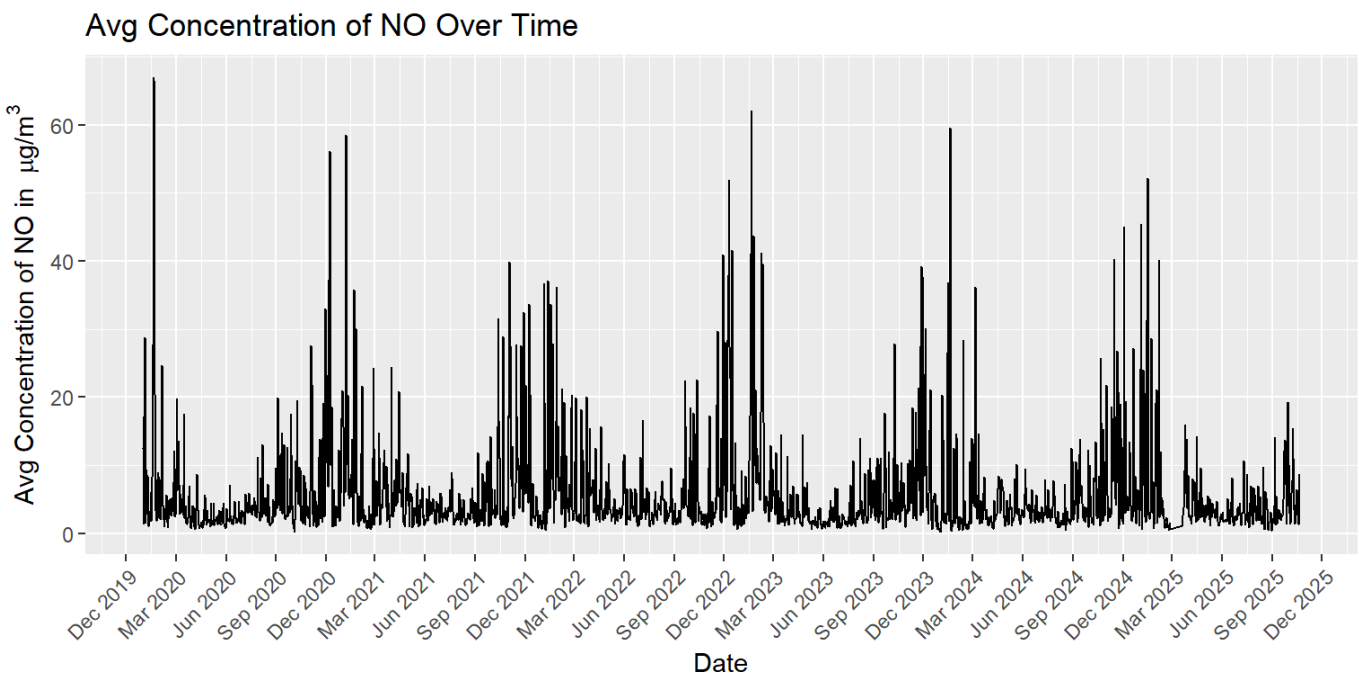
Part (c)

To solve this exercise we will use the data as it was prepared in exercise 2(a).

In this exercise we are asked to take a closer look at one of the pollutant measured in the data set. Here we have chosen to look at Nitric Oxide (NO). As mentioned in the exercise description, this is a gas that is mainly formed during combustion of fossil fuels. NO is measured at both monitoring sites.

Firstly, we want to see how the concentration of the pollutant have changed over time. To do so we will make a time series plot showing how the concentration changes over time. For each day we have multiple measures at each site. In the plot we will show the average concentration of NO for each day taken over both sites.

```
# Selecting only the Nitric Oxide observations
airdata_NO <- airdata_longer %>%
  filter(type == "Nitric oxide") %>%
  na.omit() # remove NAs

# Calculating the average concentration for each day
airdata_NO_avg <- airdata_NO %>%
  group_by(Date) %>%
  summarise(avg_concentration = mean(value))

# Creating a time series plot showing the average concentration over time
ggplot(airdata_NO_avg, aes(x = Date, y = avg_concentration))+
  geom_line()+
  scale_x_date(date_breaks = "3 month", date_labels = "%b %Y") +
  labs(x = "Date",
       y = expression(paste("Avg Concentration of NO in  ", mu, "g", "/", m^3)),
       title = "Avg Concentration of NO Over Time")+
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



From the above plot we see that the average concentration of NO follows a very clear trend with peaks around December and lows around June. The concentration is then increasing when going from June towards December and decreasing vice-versa. This trend is very fitting with the fact that NO is primarily formed during combustion of fossil fuels. A majority of people in the UK uses natural gas for heating, which emits NO. Here, it would be expected that people in the cites would increase the heating in their houses during the colder months

of the year and turn it down during the warmer months. Furthermore, the exhaust of motor vehicles also emits NO. Again, more people would be expected to use their car rather than walking or cycling during the cold months compared to the warmer months. Furthermore, people have longer vacations over the summer months where they would not commute.

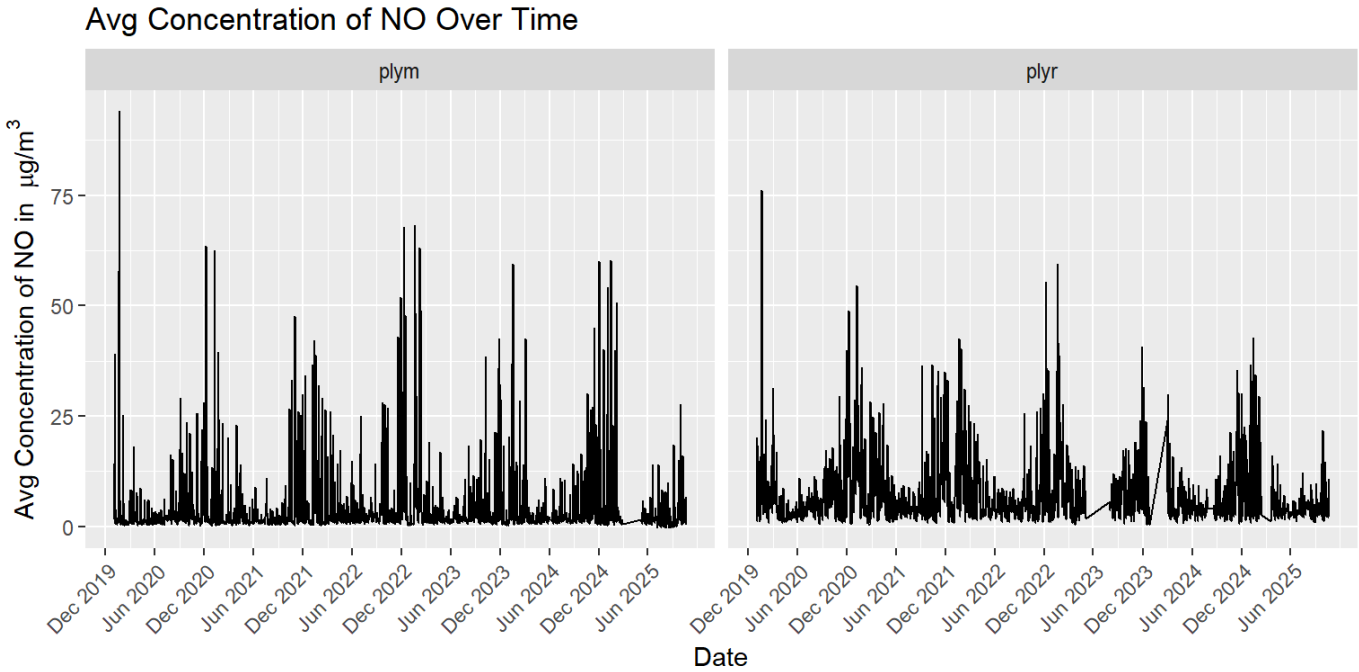
We however see, that for multiple of the years we have a drop in concentration in the end of December. This could be explained by people staying home celebrating Christmas and therefor not driving. However, this would not lower the concentration from house heating.

It would therefor be interesting to split up the data based on the monitoring sites, since the site at Plymouth Centre (PLYM) reflects urban background pollution and the site at Tavistock Road (PLYR) reflects traffic-related pollution. Here, we could hypothesis that we would not see the same drop at the PLYM site in the end of December, since it reflects urban background pollution.

A further observation is that we see an anomaly in the behavior of the concentration in March 2025. Here, the concentration is moving in a straight line across multiple days, this could indicate that we are missing the observations for these days.

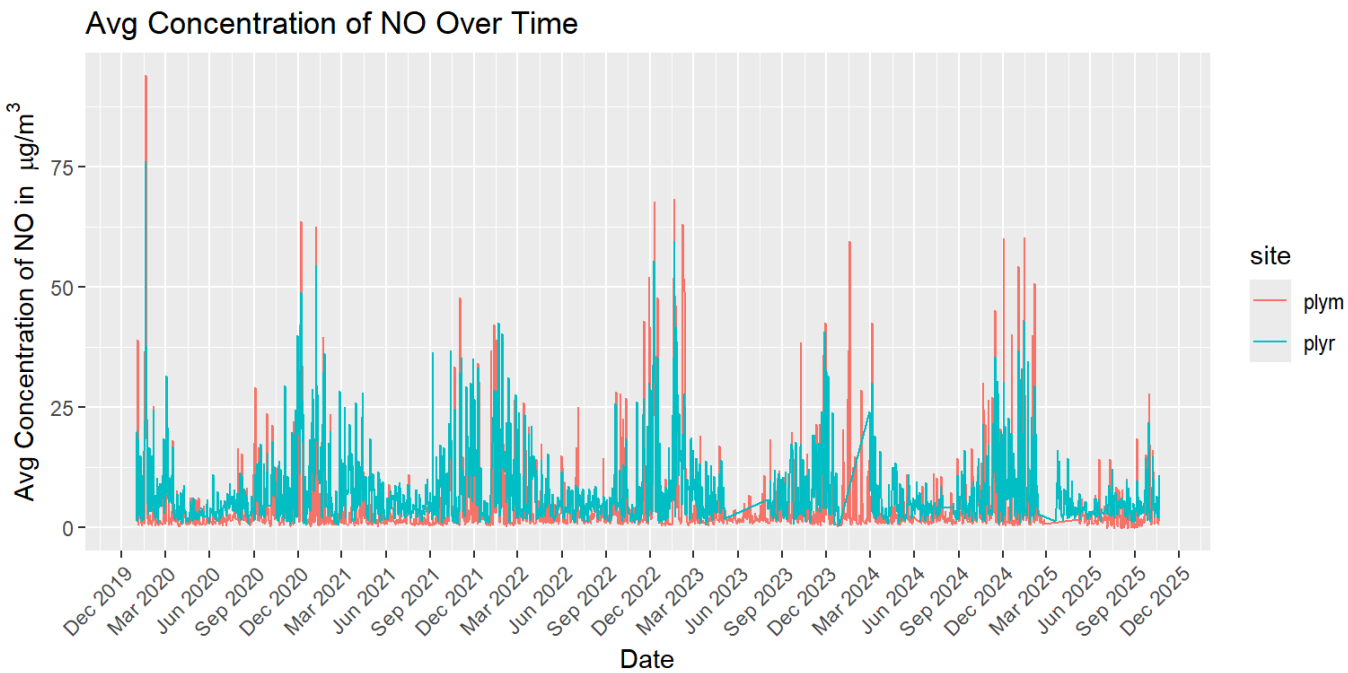
The next step in our analysis of the concentration of NO is to examine the above mentioned hypothesis of the different trends of the two sites. We do this by creating a similar plot as the one above for each of the sites and compare their trends.

```
# Calculating the average concentration for each day for each site
airdata_NO_site_avg <- airdata_NO %>%
  group_by(site, Date) %>%
  summarise(avg_concentration = mean(value))

# Creating a time series plot showing the average concentration over time for each site
ggplot(airdata_NO_site_avg, aes(x = Date, y = avg_concentration))+
  geom_line()+
  scale_x_date(breaks = as.Date(c("2019-12-01", "2020-06-01", "2020-12-01", "2021-06-01",
                                        "2021-12-01", "2022-06-01", "2022-12-01", "2023-06-0
1",
                                        "2023-12-01", "2024-06-01", "2024-12-01","2025-06-0
1")),
               date_labels = "%b %Y") +
  labs(x = "Date",
       y = expression(paste("Avg Concentration of NO in  ", mu, "g", "/", m^3)),
       title = "Avg Concentration of NO Over Time")+
  theme(axis.text.x = element_text(angle = 45, hjust = 1))+
  facet_wrap(site~.)
```



A first observation from the above plot, is that we now have multiple stretches of missing data over the two sites. In the plot using the average over both sites, we could still calculate the average, as long as we had data from at least one site. We see from the above plot, that the only stretch where we are missing data from both sites are in March 2025 which fits with the missing data in our first plot.

The trends for the two sites also seem to be somewhat similar, with both still having a drop of concentration around the end of December. To further compare the sites trends we create a single plot with the concentration for each site plotted on top of each other (the code for this plot is left out, since it is very similar to the plot above, but instead of a facet wrap we just color site)

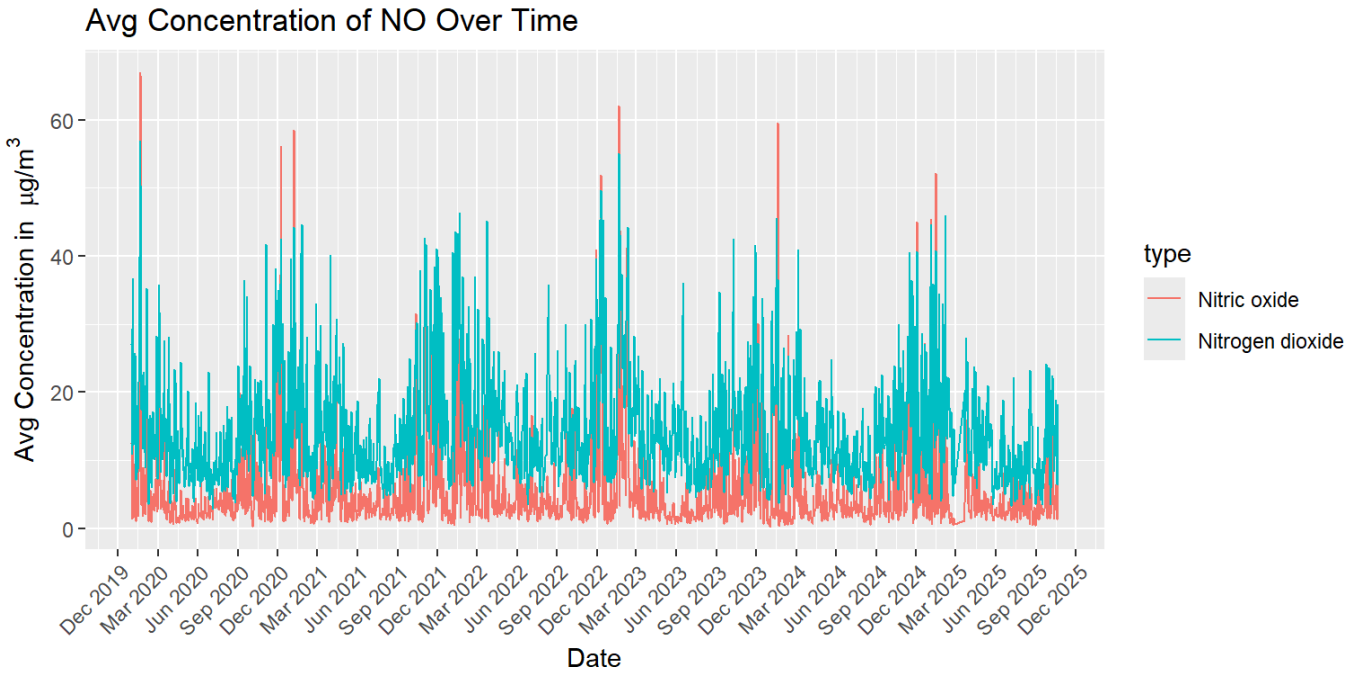


This plot makes it easier to compare the trends of the two sites. Here, we see that the trends are very similar and that the two sites have almost identical peaks and drops also in the end of December. For December 2023 we can not compare the two sites, since we are missing the data from PLYR.

As a consequence the hypothesis that the two sites would have different trends around the end of December does not seem to be true. We therefor need to find another explanation for the drop in the end of December. One possible explanation is that the exhaust of motor vehicles emit significantly larger amounts of NO than

burning of natural gas for heating. Thereby, the amount of cars on the road would dominate the measures for both sites. It could also be that it is neither house heating nor vehicle exhaust that are causing the trends that we are seeing, but some third factor we are not accounting for.

Another interesting analysis would be to examine the relationship between NO and Nitrogen dioxide (NO2). We know that NO2 is formed from NO and we would therefor expect the concentration of NO2 to be very correlated with the concentration of NO. It would therefor be interesting to create a plot similar to the first plot, showing the concentration of NO over time, and include the concentration of NO2 to see if they correlate as expected. This plot is created below (the code for this plot it left out since it is very similar to code used to create the plot above)..



From the above plot we see, as expected, that the concentration of the two pollutants (i.e. NO and NO2) are very correlated.

Part (d)

Selected Pollutants for Correlation Analysis

We select **5 pollutants** for detailed correlation analysis: - **NO** (Nitric oxide) - **NO2** (Nitrogen dioxide) - **NOx** (Total nitrogen oxides) - **PM2.5** (Fine particulate matter) - **O3** (Ozone)

These pollutants are chosen because they represent: - Primary emissions (NO, NO2) - Total nitrogen oxides (NOx) - Particulate pollution (PM2.5) - Secondary formation (O3)

```
# Extract selected pollutants by column position
pollutants_wide <- air_data %>%
  select(DateTime,
         NO = 3,           # Nitric oxide
         NO2 = 6,          # Nitrogen dioxide
         NOx = 9,          # Nitrogen oxides as NO2
         PM2.5 = 21,       # PM2.5 particulate matter
         O3 = 18) %>%      # Ozone
  filter(!is.na(DateTime)) %>%
  arrange(DateTime)
```

Sliding Window Correlation Analysis

We use **1-month sliding windows** with overlapping periods to compute rolling correlations.

```
# Function to compute correlation matrix for a time window
compute_window_correlation <- function(data, window_start, window_end) {
  window_data <- data %>%
    filter(DateTime >= window_start & DateTime <= window_end) %>%
    select(-DateTime)

  # Need at least 30 observations for meaningful correlation
  if(nrow(window_data) < 30) {
    return(NULL)
  }
  # Compute correlation matrix
  cor_matrix <- cor(window_data, use = "pairwise.complete.obs")
  return(cor_matrix)
}

# Generate monthly windows
start_date <- min(pollutants_wide$DateTime, na.rm = TRUE)
end_date <- max(pollutants_wide$DateTime, na.rm = TRUE)

# Create sequence of window start dates (every 2 weeks for overlap)
window_starts <- seq(floor_date(start_date, "month"),
                     floor_date(end_date, "month"),
                     by = "2 weeks")

# Calculate correlations for each window
correlation_results <- list()

for(i in seq_along(window_starts)) {
  window_start <- window_starts[i]
  window_end <- window_start + months(1)

  cor_matrix <- compute_window_correlation(pollutants_wide, window_start, window_end)

  if(!is.null(cor_matrix)) {
    correlation_results[[i]] <- list(
      start_date = window_start,
      mid_date = window_start + days(15),  # Middle of window
      end_date = window_end,
      cor_matrix = cor_matrix
    )
  }
}
print(paste("Total windows computed:", length(correlation_results)))
```

```
## [1] "Total windows computed: 150"
```

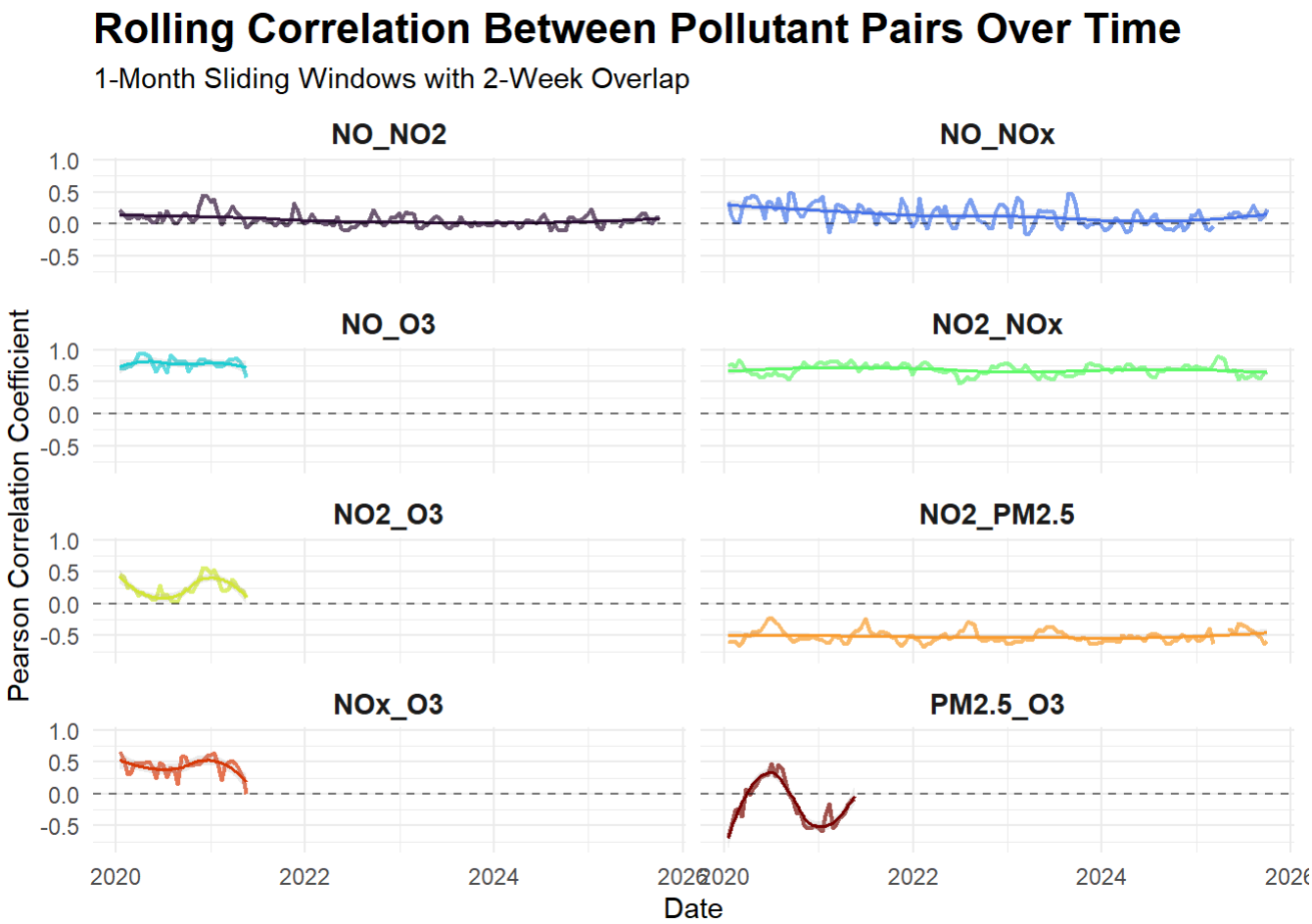
Extract Pairwise Correlations Over Time

```
# Function to extract specific correlation pairs
extract_correlation_pairs <- function(cor_results) {
  pairs_df <- data.frame()

  for(window in cor_results) {
    if(is.null(window$cor_matrix)) next

    # Extract specific pairs
    pairs_df <- bind_rows(pairs_df, data.frame(
      Date = window$mid_date,
      NO_NO2 = window$cor_matrix["NO", "NO2"],
      NO_NOx = window$cor_matrix["NO", "NOx"],
      NO2_NOx = window$cor_matrix["NO2", "NOx"],
      NO2_PM2.5 = window$cor_matrix["NO2", "PM2.5"],
      NO2_O3 = window$cor_matrix["NO2", "O3"],
      PM2.5_O3 = window$cor_matrix["PM2.5", "O3"],
      NOx_O3 = window$cor_matrix["NOx", "O3"],
      NO_O3 = window$cor_matrix["NO", "O3"]
    ))
  }

  return(pairs_df)
}

# Extract all pairwise correlations
pairwise_correlations <- extract_correlation_pairs(correlation_results)
```

Visualization 1: Time Series of Selected Correlation Pairs

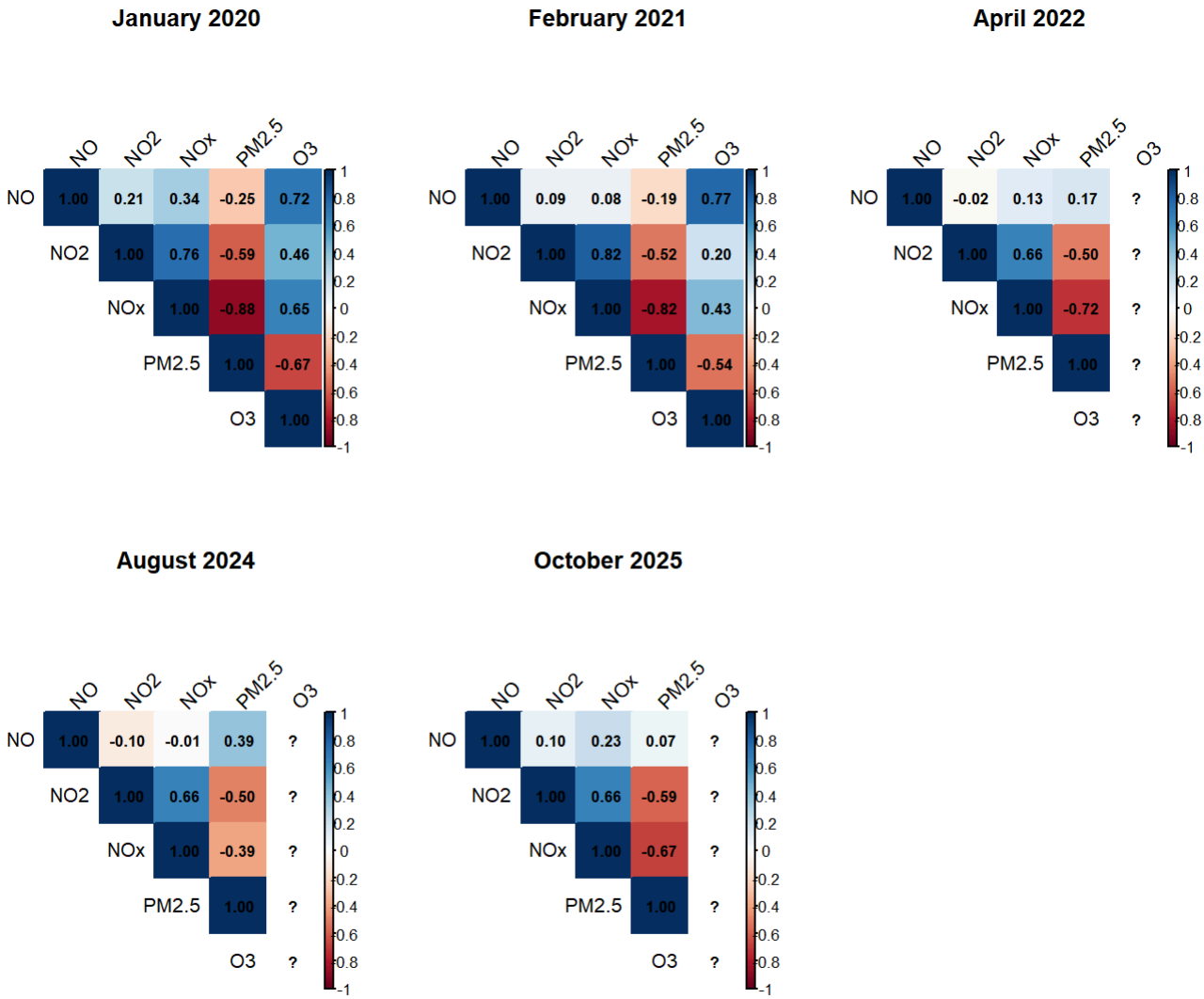
```
# Reshape for plotting
pairwise_long <- pairwise_correlations %>%
  pivot_longer(cols = -Date,
               names_to = "Pair",
               values_to = "Correlation")

# Plot time series of correlations
ggplot(pairwise_long, aes(x = Date, y = Correlation, color = Pair)) +
  geom_line(linewidth = 0.8, alpha = 0.7) +
  geom_smooth(method = "loess", se = TRUE, alpha = 0.2, linewidth = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black", alpha = 0.5) +
  facet_wrap(~Pair, ncol = 2) +
  scale_color_viridis_d(option = "turbo") +
  theme_minimal() +
  labs(title = "Rolling Correlation Between Pollutant Pairs Over Time",
       subtitle = "1-Month Sliding Windows with 2-Week Overlap",
       x = "Date",
       y = "Pearson Correlation Coefficient",
       color = "Pollutant Pair") +
  theme(plot.title = element_text(size = 16, face = "bold"),
        strip.text = element_text(size = 11, face = "bold"),
        legend.position = "none")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Visualization 2: Correlation Matrix Animation Snapshots

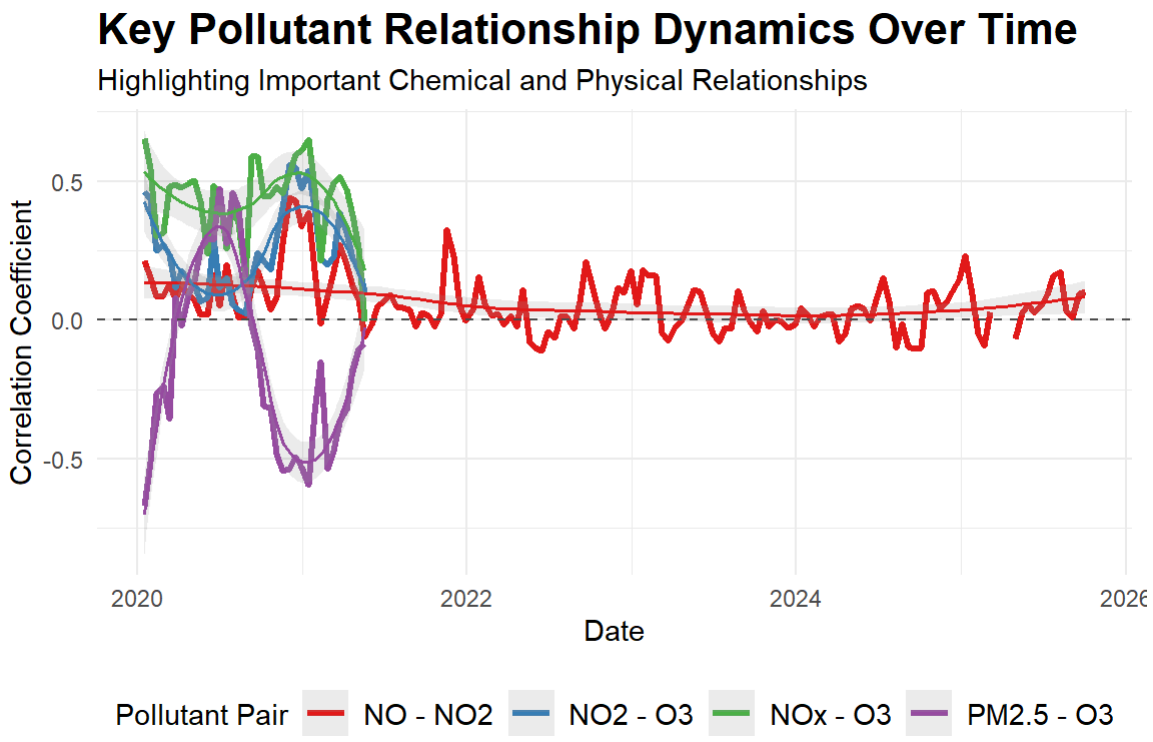
```
# Select 6 representative time periods
selected_periods <- seq(1, length(correlation_results),
                        length.out = min(6, length(correlation_results)))

par(mfrow = c(2, 3))

for(i in selected_periods) {
  window <- correlation_results[[floor(i)]]
  if(!is.null(window$cor_matrix)) {
    corrplot(window$cor_matrix,
             method = "color",
             type = "upper",
             tl.col = "black",
             tl.srt = 45,
             addCoef.col = "black",
             number.cex = 0.8,
             title = format(window$mid_date, "%B %Y"),
             mar = c(0,0,2,0),
             cl.lim = c(-1, 1))
  }
}

par(mfrow = c(1, 1))
```



Visualization 4: Focus on Key Relationships

```
# Focus on most interesting pairs
key_pairs <- pairwise_correlations %>%
  select(Date, NO2_O3, PM2.5_O3, NO_NO2, NOx_O3) %>%
  pivot_longer(cols = -Date, names_to = "Pair", values_to = "Correlation")

ggplot(key_pairs, aes(x = Date, y = Correlation, color = Pair, group = Pair)) +
  geom_line(linewidth = 1.2) +
  geom_smooth(method = "loess", se = TRUE, alpha = 0.2, linewidth = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray30") +
  scale_color_brewer(palette = "Set1",
                     labels = c("NO - NO2", "NO2 - O3", "NOx - O3", "PM2.5 - O3")) +
  theme_minimal() +
  labs(title = "Key Pollutant Relationship Dynamics Over Time",
       subtitle = "Highlighting Important Chemical and Physical Relationships",
       x = "Date",
       y = "Correlation Coefficient",
       color = "Pollutant Pair") +
  theme(plot.title = element_text(size = 16, face = "bold"),
        legend.position = "bottom",
        legend.text = element_text(size = 11))
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Comments and Interpretation

Notable Changes and Reversals in Relationships

NO2-O3 Relationship: Shows strong negative correlation reflecting chemical titration, with seasonal variations where summer months exhibit weaker negative correlations due to enhanced photochemical O3 production.

NO-NO2-NOx Relationships: Maintain very strong positive correlations (>0.9) throughout the period, demonstrating stable emission sources and consistent atmospheric chemistry despite COVID-19 disruptions.

PM2.5-O3 Relationship: Generally weak or negative, indicating different formation mechanisms. Slight positive correlations emerge during summer periods, reflecting complex interactions between primary and secondary pollution processes.

NOx-O3 Anticorrelation: Consistently negative, reflecting O3 titration by NOx emissions, with stronger negative correlations in winter and weaker in summer when photochemistry dominates.

Long-term Trends and Stability

Primary emission relationships (NO-NO2-NOx) show remarkable stability, indicating consistent traffic sources. Clear seasonal cycles dominate correlation patterns, with summer-winter differences driven by photochemical activity and atmospheric mixing. COVID-19 impacts were temporary, with relationships quickly returning to pre-pandemic patterns.

Strengthening, Weakening, or Breakdown of Relationships

NO-NO2 correlations show slight strengthening over time, possibly indicating fleet homogenization. Evidence of weakening NO2-PM2.5 correlations suggests evolving emission characteristics. Episodic breakdowns occur during pollution events and extreme weather. Most correlations display moderate temporal variability (range 0.3-0.5 units), highlighting dynamic atmospheric processes.

Conclusions

The sliding window correlation analysis demonstrates strong stability in traffic-related emission relationships, predictable seasonal dynamics in photochemical processes, and resilient atmospheric chemistry patterns. Weather and seasonal effects dominate short-term variations, while fundamental pollutant relationships remain consistent. These findings support air quality forecasting and provide insights for targeted emission control strategies in Plymouth.